Recent Trends in Deep Learning Based Natural Language Processing

Tom Young[†≡], Devamanyu Hazarika[‡≡], Soujanya Poria[⊕≡], Erik Cambria[▽*]

[†] School of Information and Electronics, Beijing Institute of Technology, China
[‡] School of Computing, National University of Singapore, Singapore
[⊕] Temasek Laboratories, Nanyang Technological University, Singapore
[▽] School of Computer Science and Engineering, Nanyang Technological University, Singapore

Abstract

Deep learning methods employ multiple processing layers to learn hierarchical representations of data, and have produced state-of-the-art results in many domains. Recently, a variety of model designs and methods have blossomed in the context of natural language processing (NLP). In this paper, we review significant deep learning related models and methods that have been employed for numerous NLP tasks and provide a walk-through of their evolution. We also summarize, compare and contrast the various models and put forward a detailed understanding of the past, present and future of deep learning in NLP.

Index Terms

Natural Language Processing, Deep Learning, Word2Vec, Attention, Recurrent Neural Networks, Convolutional Neural Networks, LSTM, Sentiment Analysis, Question Answering, Dialogue Systems, Parsing, Named-Entity Recognition, POS Tagging, Semantic Role Labeling

I. Introduction

Natural language processing (NLP) is a theory-motivated range of computational techniques for the automatic analysis and representation of human language. NLP research has evolved from the era of punch cards and batch processing, in which the analysis of a sentence could take up to 7 minutes, to the era of Google and the likes of it, in which millions of webpages can be processed in less than a second [1]. NLP enables computers to perform a wide range of natural language related tasks at all levels, ranging from parsing and part-of-speech (POS) tagging, to machine translation and dialogue systems.

Deep learning architectures and algorithms have already made impressive advances in fields such as computer vision and pattern recognition. Following this trend, recent NLP research is now increasingly focusing on the use of new deep learning methods (see Figure 1). For decades, machine learning approaches targeting NLP problems have been based on shallow models (e.g., SVM and logistic regression) trained on very high dimensional and sparse features. In the last few years, neural networks based on dense vector representations have been producing superior results on various NLP tasks. This trend is sparked by the success of word embeddings [2, 3] and deep learning methods [4]. Deep learning enables multi-level automatic feature representation learning. In contrast, traditional machine learning based NLP systems liaise heavily on hand-crafted features. Such hand-crafted features are time-consuming and often incomplete.

Collobert et al. [5] demonstrated that a simple deep learning framework outperforms most state-of-the-art approaches in several NLP tasks such as named-entity recognition (NER), semantic role labeling (SRL), and POS tagging. Since then, numerous complex deep learning based algorithms have been proposed to solve difficult NLP tasks. We review major deep learning related models and methods applied to natural language tasks such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and recursive neural networks. We also discuss memory-augmenting strategies, attention mechanisms and how unsupervised models, reinforcement learning methods and recently, deep generative models have been employed for language-related tasks.

To the best of our knowledge, this work is the first of its type to comprehensively cover the most popular deep learning methods in NLP research today [1]. The work by Goldberg [6] only presented the basic principles for applying neural networks to NLP in a tutorial manner. We believe this paper will give readers a more comprehensive idea of current practices in this domain.

The structure of the paper is as follows: Section II introduces the concept of distributed representation, the basis of sophisticated deep learning models; next, Sections III, IV, and V discuss popular models such as convolutional, recurrent, and recursive neural networks, as well as their use in various NLP tasks; following, Section VI lists recent applications of reinforcement learning in NLP and new developments in unsupervised sentence representation learning; later, Section VII

≡ means authors contributed equally
* Corresponding author (e-mail: cambria@ntu.edu.sg)
[1] We intend to update this article with time as and when significant advances are proposed and used by the community

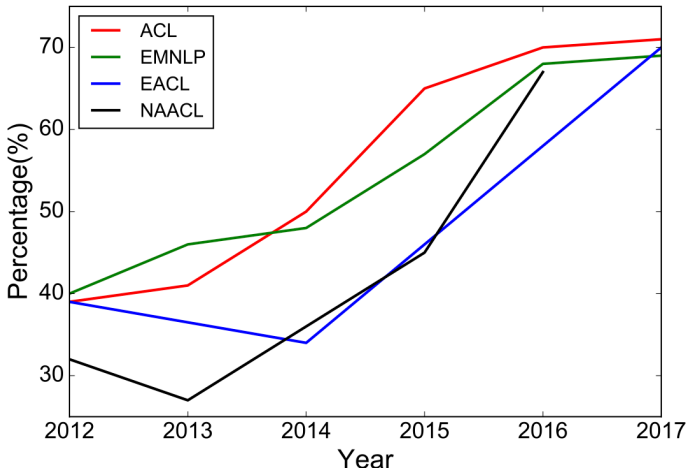


Fig. 1: Percentage of deep learning papers in ACL, EMNLP, EACL, NAACL over the last 6 years (long papers).

illustrates the recent trend of coupling deep learning models with memory modules; finally, Section VIII summarizes the performance of a series of deep learning methods on standard datasets about major NLP topics.

II. Distributed Representation

Statistical NLP has emerged as the primary option for modeling complex natural language tasks. However, in its beginning, it often used to suffer from the notorious *curse of dimensionality* while learning joint probability functions of language models. This led to the motivation of learning distributed representations of words existing in low-dimensional space [7].

A. *Word Embeddings*

Distributional vectors or word embeddings (Fig. 2) essentially follow the distributional hypothesis, according to which words with similar meanings tend to occur in similar context. Thus, these vectors try to capture the characteristics of the neighbors of a word. The main advantage of distributional vectors is that they capture similarity between words. Measuring similarity between vectors is possible, using measures such as cosine similarity. Word embeddings are often used as the first data processing layer in a deep learning model. Typically, word embeddings are pre-trained by optimizing an auxiliary objective in a large unlabeled corpus, such as predicting a word based on its context [8, 3], where the learned word vectors can capture general syntactical and semantic information. Thus, these embeddings have proven to be efficient in capturing context similarity, analogies and due to its smaller dimensionality, are fast and efficient in processing core NLP tasks.

Over the years, the models that create such embeddings have been shallow neural networks and there has not been need for deep networks to create good embeddings. However, deep learning based NLP models invariably represent their words, phrases and even sentences using these embeddings. This is in fact a major difference between traditional word count based models and deep learning based models. Word embeddings have been responsible for state-of-the-art results in a wide range of NLP tasks [9, 10, 11, 12].

For example, Glorot et al. [13] used embeddings along with stacked denoising autoencoders for domain adaptation in sentiment classification and Hermann and Blunsom [14] presented combinatory categorial autoencoders to learn the compositionality of sentence. Their wide usage across the recent literature shows their effectiveness and importance in any deep learning model performing a NLP task.

Distributed representations (embeddings) are mainly learned through context. During 1990s, several research developments [15] marked the foundations of research in distributional semantics. A more detailed summary of these early trends is

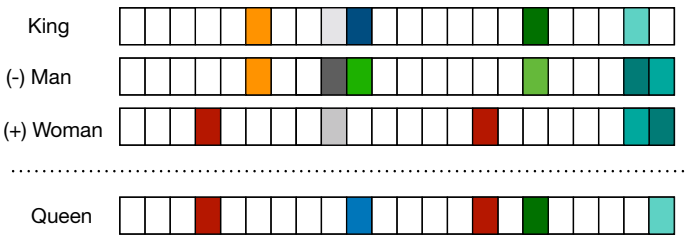


Fig. 2: Distributional vectors represented by a $\mathbf{D}$-dimensional vector where $\mathbf{D} << \mathbf{V}$, where $\mathbf{V}$ is size of Vocabulary. Figure Source: http://veredshwartz.blogspot.sg.

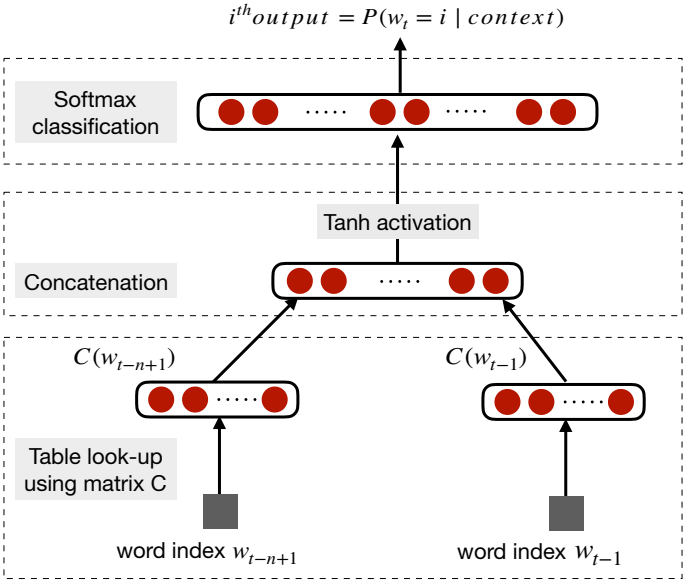


Fig. 3: Neural Language Model (Figure reproduced from Bengio et al. [7]). $C(i)$ is the i^{th} word embedding.

provided in [16, 17]. Later developments were adaptations of these early works, which led to creation of topic models like latent Dirichlet allocation [18] and language models [7]. These works laid out the foundations of representation learning in natural language.

In 2003, Bengio et al. [7] proposed a neural language model which learned distributed representations for words (Fig. 3). Authors argued that these word representations, once compiled into sentence representations using joint probability of word sequences, achieved an exponential number of semantically neighboring sentences. This, in turn, helped in generalization since unseen sentences could now gather higher confidence if word sequences with similar words (in respect to nearby word representation) were already seen.

Collobert and Weston [19] were the first work to show the utility of pre-trained word embeddings. They proposed a neural network architecture that forms the foundation to many current approaches. The work also establishes word embeddings as a useful tool for NLP tasks. However, the immense popularization of word embeddings was arguably due to Mikolov et al. [3] who proposed the continuous bag-of-words (CBOW) and skip-gram models to efficiently construct high-quality distributed vector representations. Propelling their popularity was the unexpected side effect of the vectors exhibiting compositionality, i.e., adding two word vectors results in a vector that is a semantic composite of the individual words, e.g., 'man' + 'royal' = 'king'. The theoretical justification for this behavior was recently given by Gittens et al. [20], which stated that compositionality is seen only when certain assumptions are held, e.g., the assumption that words need to be uniformly distributed in the embedding space.

Glove by Pennington et al. [21] is another famous word embedding method which is essentially a "count-based" model. Here, the word co-occurrence count matrix is pre-processed by normalizing the counts and log-smoothing operation. This matrix is then factorized to get lower dimensional representations which is done by minimizing a "reconstruction loss".

Below, we provide a brief description of the word2vec method proposed by Mikolov et al. [3].

B. Word2vec

Word embeddings were revolutionized by Mikolov et al. [8, 3] who proposed the CBOW and skip-gram models. CBOW computes the conditional probability of a target word given the context words surrounding it across a window of size k. On the other hand, the skip-gram model does the exact opposite of the CBOW model, by predicting the surrounding context words given the central target word. The context words are assumed to be located symmetrically to the target words within a distance equal to the window size in both directions. In unsupervised settings, the word embedding dimension is determined by the accuracy of prediction. As the embedding dimension increases, the accuracy of prediction also increases until it converges at some point, which is considered the optimal embedding dimension as it is the shortest without compromising accuracy.

Let us consider a simplified version of the CBOW model where only one word is considered in the context. This essentially replicates a bigram language model.

As shown in Fig. 4, the CBOW model is a simple fully connected neural network with one hidden layer. The input layer, which takes the one-hot vector of context word has V neurons while the hidden layer has N neurons. The output layer is softmax probability over all words in the vocabulary. The layers are connected by weight matrix $\mathbf{W} \in \mathcal{R}^{V \times N}$ and $\mathbf{W}' \in \mathcal{R}^{H \times V}$,

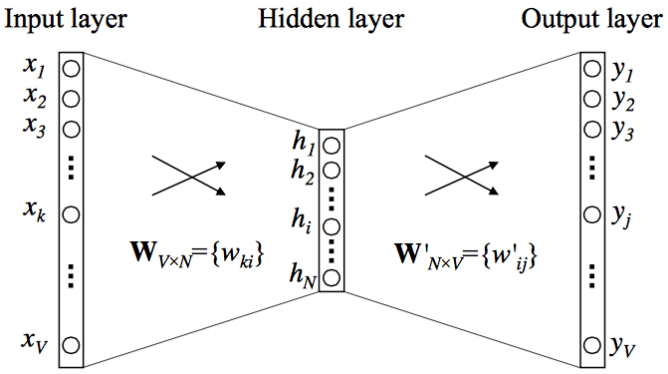


Fig. 4: Model for CBOW (Figure source: Rong [22])

respectively. Each word from the vocabulary is finally represented as two learned vectors $\mathbf{v_c}$ and $\mathbf{v_w}$, corresponding to context and target word representations, respectively. Thus, k^{th} word in the vocabulary will have

$$\mathbf{v_c} = \mathbf{W}_{(\mathbf{k},.)} \quad and \quad \mathbf{v_w} = \mathbf{W}^{'}_{(.,\mathbf{k})} \tag{1}$$

Overall, for any word w_i with given context word c as input,

$$p\left(\frac{w_i}{c}\right) = \mathbf{y_i} = \frac{e^{u_i}}{\sum_{i=1}^{V} e^{u_i}} \quad where, \quad u_i = \mathbf{v^T_{w_i}}.\mathbf{v_c} \tag{2}$$

The parameters $\theta = \{\mathbf{v_w}, \mathbf{v_c}\}_{w,c \in \text{Vocab}}$ are learned by defining the objective function as the log-likelihood and finding its gradient as

$$\mathbf{l}(\theta) = \sum_{\mathbf{w} \in \mathbf{Vocab}} \mathbf{log}\left(\mathbf{p}\left(\frac{\mathbf{w}}{\mathbf{c}}\right)\right) \tag{3}$$

$$\frac{\partial \mathbf{l}(\theta)}{\partial \mathbf{v_w}} = \mathbf{v_c}\left(\mathbf{1} - \mathbf{p}\left(\frac{\mathbf{w}}{\mathbf{c}}\right)\right) \tag{4}$$

In the general CBOW model, all the one-hot vectors of context words are taken as input simultaneously, i.e,

$$\mathbf{h} = \mathbf{W^T}(\mathbf{x_1} + \mathbf{x_2} + ... + \mathbf{x_c}) \tag{5}$$

One limitation of individual word embeddings is their inability to represent phrases [3], where the combination of two or more words – e.g., idioms like "hot potato" or named entities such as "Boston Globe' – does not represent the combination of meanings of individual words. One solution to this problem, as explored by Mikolov et al. [3], is to identify such phrases based on word co-occurrence and train embeddings for them separately. Later methods have explored directly learning n-gram embeddings from unlabeled data [23].

Another limitation comes from learning embeddings based only on a small window of surrounding words, sometimes words such as *good* and *bad* share almost the same embedding [24], which is problematic if used in tasks such as sentiment analysis [25]. At times these embeddings cluster semantically-similar words which have opposing sentiment polarities. This leads the downstream model used for the sentiment analysis task to be unable to identify this contrasting polarities leading to poor performance. Tang et al. [26] addressed this problem by proposing sentiment specific word embedding (SSWE). Authors incorporated the supervised sentiment polarity of text in their loss functions while learning the embeddings.

A general caveat for word embeddings is that they are highly dependent on the applications in which it is used. Labutov and Lipson [27] proposed task specific embeddings which retrain the word embeddings to align them in the current task space. This is very important as training embeddings from scratch requires large amount of time and resource. Mikolov et al. [8] tried to address this issue by proposing *negative sampling* which does frequency-based sampling of negative terms while training the word2vec model.

Traditional word embedding algorithms assign a distinct vector to each word. This makes them unable to account for polysemy. In a recent work, Upadhyay et al. [28] provided an innovative way to address this deficit. The authors leveraged multilingual parallel data to learn multi-sense word embeddings. For example, the English word bank, when translated to French provides two different words: *banc* and *banque* representing financial and geographical meanings, respectively. Such multilingual distributional information helped them in accounting for polysemy.

Table I provides a directory of existing frameworks that are frequently used for creating embeddings which are further incorporated into deep learning models.

Framework	Language	URL
S-Space	Java	https://github.com/fozziethebeat/S-Space
Semanticvectors	Java	https://github.com/semanticvectors/
Gensim	Python	https://radimrehurek.com/gensim/
Pydsm	Python	https://github.com/jimmycallin/pydsm
Dissect	Python	http://clic.cimec.unitn.it/composes/toolkit/
FastText	Python	https://fasttext.cc/
Elmo	Python	https://tfhub.dev/google/elmo/2

TABLE I: Frameworks providing word embedding tools and methods.

C. *Character Embeddings*

Word embeddings are able to capture syntactic and semantic information, yet for tasks such as POS-tagging and NER, intra-word morphological and shape information can also be very useful. Generally speaking, building natural language understanding systems at the character level has attracted certain research attention [29, 30, 31, 32]. Better results on morphologically rich languages are reported in certain NLP tasks. Santos and Guimaraes [31] applied character-level representations, along with word embeddings for NER, achieving state-of-the-art results in Portuguese and Spanish corpora. Kim et al. [29] showed positive results on building a neural language model using only character embeddings. Ma et al. [33] exploited several embeddings, including character trigrams, to incorporate prototypical and hierarchical information for learning pre-trained label embeddings in the context of NER.

A common phenomenon for languages with large vocabularies is the unknown word issue, also known as *out-of-vocabulary* (OOV) words. Character embeddings naturally deal with it since each word is considered as no more than a composition of individual letters. In languages where text is not composed of separated words but individual characters and the semantic meaning of words map to its compositional characters (such as Chinese), building systems at the character level is a natural choice to avoid word segmentation [34]. Thus, works employing deep learning applications on such languages tend to prefer character embeddings over word vectors [35]. For example, Peng et al. [36] proved that radical-level processing could greatly improve sentiment classification performance. In particular, the authors proposed two types of Chinese radical-based hierarchical embeddings, which incorporate not only semantics at radical and character level, but also sentiment information. Bojanowski et al. [37] also tried to improve the representation of words by using character-level information in morphologically-rich languages. They approached the skip-gram method by representing words as bag-of-character n-grams. Their work thus had the effectiveness of the skip-gram model along with addressing some persistent issues of word embeddings. The method was also fast, which allowed training models on large corpora quickly. Popularly known as *FastText*, such a method stands out over previous methods in terms of speed, scalability, and effectiveness.

Apart from character embeddings, different approaches have been proposed for OOV handling. Herbelot and Baroni [38] provided on-the-fly OOV handling by initializing the unknown words as the sum of the context words and refining these words with a high learning rate. However, their approach is yet to be tested on typical NLP tasks. Pinter et al. [39] provided an interesting approach of training a character-based model to recreate pre-trained embeddings. This allowed them to learn a compositional mapping form character to word embedding, thus tackling the OOV problem.

Despite the ever growing popularity of distributional vectors, recent discussions on their relevance in the long run have cropped up. For example, Lucy and Gauthier [40] has recently tried to evaluate how well the word vectors capture the necessary facets of conceptual meaning. The authors have discovered severe limitations in perceptual understanding of the concepts behind the words, which cannot be inferred from distributional semantics alone. A possible direction for mitigating these deficiencies will be grounded learning, which has been gaining popularity in this research domain.

D. *Contextualized Word Embeddings*

The quality of word representations is generally gauged by its ability to encode syntactical information and handle polysemic behavior (or word senses). These properties result in improved semantic word representations. Recent approaches in this area encode such information into its embeddings by leveraging the context. These methods provide deeper networks that calculate word representations as a function of its context.

Traditional word embedding methods such as Word2Vec and Glove consider all the sentences where a word is present in order to create a global vector representation of that word. However, a word can have completely different senses or meanings in the contexts. For example, lets consider these two sentences - 1) “The *bank* will not be accepting cash on Saturdays” 2) “The river overflowed the *bank*.”. The word senses of *bank* are different in these two sentences depending on its context. Reasonably, one might want two different vector representations of the word *bank* based on its two different word senses. The new class of models adopt this reasoning by diverging from the concept of global word representations and proposing contextual word embeddings instead.

Embedding from Language Model (ELMo) [41] is one such method that provides deep contextual embeddings. ELMo produces word embeddings for each context where the word is used, thus allowing different representations for varying senses

of the same word. Specifically, for *N* different sentences where a word *w* is present, ELMo generates *N* different representations of w i.e., $\boldsymbol{w}_1, \boldsymbol{w}_2, ;\boldsymbol{w}_N$.

The mechanism of ELMo is based on the representation obtained from a bidirectional language model. A bidirectional language model (biLM) constitutes of two language models (LM) 1) *forward LM* and 2) *backward LM*. A forward LM takes input representation $\boldsymbol{x}_k^{LM}$ for each of the k^{th} token and passes it through L layers of forward LSTM to get representations $\overrightarrow{\boldsymbol{h}}_{k,j}^{LM}$ where $j = 1, \ldots, L$. Each of these representations, being hidden representations of recurrent neural networks, is context dependent. A forward LM can be seen as a method to model the joint probability of a sequence of tokens: $p\left(t_1, t_2, \ldots, t_N\right) = \prod_{k=1}^{N} p\left(t_k | t_1, t_2, \ldots, t_{k-1}\right)$. At a timestep $k-1$ the forward LM predicts the next token t_k given the previous observed tokens $t_1, t_2, ..., t_k$. This is typically achieved by placing a softmax layer on top of the final LSTM in a forward LM. On the other hand, a backward LM models the same joint probability of the sequence by predicting the previous token given the future tokens: $p\left(t_1, t_2, \ldots, t_N\right) = \prod_{k=1}^{N} p\left(t_k | t_{k+1}, t_{k+2}, \ldots, t_N\right)$. In other words, a backward LM is similar to forward LM which processes a sequence with the order being reversed. The training of the biLM model involves modeling the log-likelihood of both the sentence orientations. Finally, hidden representations from both LMs are concetenated to compose the final token vectors [42].

For each tokem, ELMo extracts the intermediate layer representations from the biLM and performs a linear combination based on the given downstream task. A L-layer biLM contains $2L + 1$ set of representations as shown below -

$$\begin{aligned} R_k &= \left\{ \mathbf{x}_k^{LM}, \overrightarrow{\mathbf{h}}_{k,j}^{LM}, \overleftarrow{\mathbf{h}}_{k,j}^{LM} | j = 1, \ldots, L \right\} \\ &= \left\{ \mathbf{h}_{k,j}^{LM} | j = 0, \ldots, L \right\} \end{aligned} \tag{6}$$

Here, $h_{k,0}^{LM}$ is the token representation at the lowest level. One can use either character or word embeddings to initialize $h_{k,0}^{LM}$. For other values of j,

$$h_{k,j}^{LM} = \left[\overrightarrow{\mathbf{h}}_{k,j}^{LM}, \overleftarrow{\mathbf{h}}_{k,j}^{LM} \right] \ \forall j = 1, \ldots, L. \tag{7}$$

ELMo flattens all layers in R in a single vector such that -

$$\mathbf{ELMo}_k^{task} = E\left(R_k; \Theta^{task}\right) = \gamma^{task} \sum_{j=0}^{L} s_j^{task} \mathbf{h}_{k,j}^{LM} \tag{8}$$

In Eq. 8, s_j^{task} is the softmax-normalized weight vector to combine the representations of different layers. γ^{task} is a hyper-parameter which helps in optimization and task specific scaling of the ELMo representation. ELMo produces varied word representations for the same word in different sentences. According to Peters et al. [41], it is always beneficial to combine ELMo word representations with standard global word representations like Glove and Word2Vec.

Off-late, there has been a surge of interest in pre-trained language models for myriad of natural language tasks [43]. Language modeling is chosen as the pre-training objective as it is widely considered to incorporate multiple traits of natual language understanding and generation. A good language model requires learning complex characteristics of language involving syntactical properties and also semantical coherence. Thus, it is believed that unsupervised training on such objectives would infuse better linguistic knowledge into the networks than random initialization. The *generative pre-training* and *discriminative fine-tuning* procedure is also desirable as the pre-training is unsupervised and does not require any manual labeling.

Radford et al. [44] proposed similar pre-trained model, the OpenAI-GPT, by adapting the *Transformer* (see section IV-E). Recently, Devlin et al. [45] proposed BERT which utilizes a transformer network to pre-train a language model for extracting contextual word embeddings. Unlike ELMo and OpenAI-GPT, BERT uses different pre-training tasks for language modeling. In one of the tasks, BERT randomly masks a percentage of words in the sentences and only predicts those masked words. In the other task, BERT predicts the next sentence given a sentence. This task in particular tries to model the relationship among two sentences which is supposedly not captured by traditional bidirectional language models. Consequently, this particular pre-training scheme helps BERT to outperform state-of-the-art techniques by a large margin on key NLP tasks such as QA, Natural Language Inference (NLI) where understanding relation among two sentences is very important. We discuss the impact of these proposed models and the performance achieved by them in section VIII-I.

The described approaches for contextual word embeddings promises better quality representations for words. The pre-trained deep language models also provide a headstart for downstream tasks in the form of transfer learning. This approach has been extremely popular in computer vision tasks. Whether there would be similar trends in the NLP community, where researchers and practitioners would prefer such models over traditional variants remains to be seen in the future.

III. Convolutional Neural Networks

Following the popularization of word embeddings and its ability to represent words in a distributed space, the need arose for an effective feature function that extracts higher-level features from constituting words or n-grams. These abstract features would then be used for numerous NLP tasks such as sentiment analysis, summarization, machine translation, and question answering (QA). CNNs turned out to be the natural choice given their effectiveness in computer vision tasks [46, 47, 48].

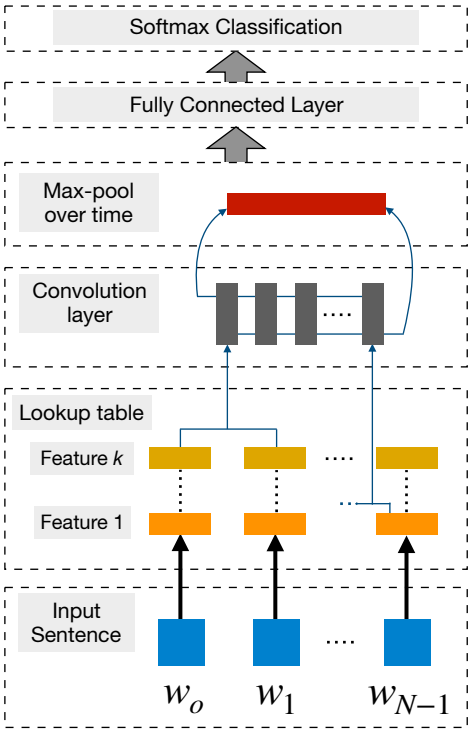


Fig. 5: CNN framework used to perform word wise class prediction (Figure source: Collobert and Weston [19])

The use of CNNs for sentence modeling traces back to Collobert and Weston [19]. This work used multi-task learning to output multiple predictions for NLP tasks such as POS tags, chunks, named-entity tags, semantic roles, semantically-similar words and a language model. A look-up table was used to transform each word into a vector of user-defined dimensions. Thus, an input sequence $\{s_1, s_2, ...s_n\}$ of n words was transformed into a series of vectors $\{\boldsymbol{w}_{s_1}, \boldsymbol{w}_{s_2}, ...\boldsymbol{w}_{s_n}\}$ by applying the look-up table to each of its words (Fig. 5).

This can be thought of as a primitive word embedding method whose weights were learned in the training of the network. In [5], Collobert extended his work to propose a general CNN-based framework to solve a plethora of NLP tasks. Both these works triggered a huge popularization of CNNs amongst NLP researchers. Given that CNNs had already shown their mettle for computer vision tasks, it was easier for people to believe in their performance.

CNNs have the ability to extract salient n-gram features from the input sentence to create an informative latent semantic representation of the sentence for downstream tasks. This application was pioneered by Collobert et al. [5], Kalchbrenner et al. [49], Kim [50], which led to a huge proliferation of CNN-based networks in the succeeding literature. Below, we describe the working of a simple CNN-based sentence modeling network:

A. Basic CNN

1) Sentence Modeling: For each sentence, let $\mathbf{w_i} \in \mathcal{R}^d$ represent the word embedding for the i^{th} word in the sentence, where d is the dimension of the word embedding. Given that a sentence has n words, the sentence can now be represented as an embedding matrix $\mathbf{W} \in \mathcal{R}^{n \times d}$. Fig. 6 depicts such a sentence as an input to the CNN framework.

Let $\mathbf{w_{i:i+j}}$ refer to the concatenation of vectors $\mathbf{w_i}, \mathbf{w_{i+1}}, ...\mathbf{w_j}$. Convolution is performed on this input embedding layer. It involves a *filter* $\mathbf{k} \in \mathcal{R}^{hd}$ which is applied to a window of h words to produce a new feature. For example, a feature c_i is generated using the window of words $\mathbf{w_{i:i+h-1}}$ by

$$c_i = f(\mathbf{w_{i:i+h-1}}.\mathbf{k}^T + b) \tag{9}$$

Here, $b \in \mathcal{R}$ is the bias term and f is a non-linear activation function, for example the hyperbolic tangent. The filter k is applied to all possible windows using the same weights to create the feature map.

$$c = [c_1, c_2, ..., c_{n-h+1}] \tag{10}$$

In a CNN, a number of convolutional filters, also called kernels (typically hundreds), of different widths slide over the entire word embedding matrix. Each kernel extracts a specific pattern of n-gram. A convolution layer is usually followed by a max-pooling strategy, $\hat{c} = max\{c\}$, which subsamples the input typically by applying a max operation on each filter. This strategy has two primary reasons.

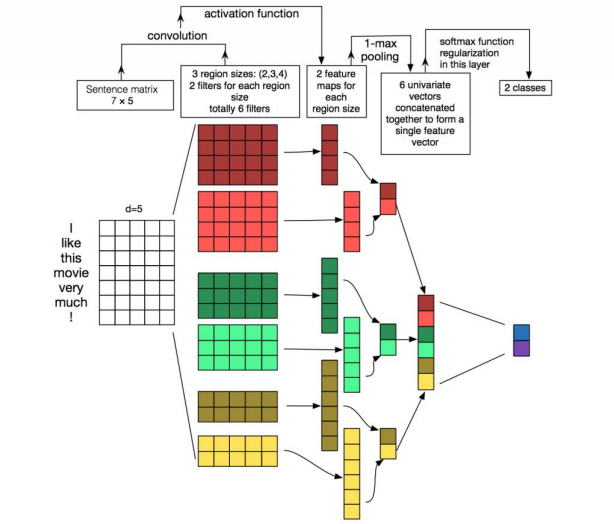


Fig. 6: CNN modeling on text (Figure source: Zhang and Wallace [51])

Firstly, max pooling provides a fixed-length output which is generally required for classification. Thus, regardless the size of the filters, max pooling always maps the input to a fixed dimension of outputs. Secondly, it reduces the output's dimensionality while keeping the most salient n-gram features across the whole sentence. This is done in a translation invariant manner where each filter is now able to extract a particular feature (e.g., negations) from anywhere in the sentence and add it to the final sentence representation.

The word embeddings can be initialized randomly or pre-trained on a large unlabeled corpora (as in Section II). The latter option is sometimes found beneficial to performance, especially when the amount of labeled data is limited [50]. This combination of convolution layer followed by max pooling is often stacked to create deep CNN networks. These sequential convolutions help in improved mining of the sentence to grasp a truly abstract representations comprising rich semantic information. The kernels through deeper convolutions cover a larger part of the sentence until finally covering it fully and creating a global summarization of the sentence features.

2) Window Approach: The above-mentioned architecture allows for modeling of complete sentences into sentence representations. However, many NLP tasks, such as NER, POS tagging, and SRL, require word-based predictions. To adapt CNNs for such tasks, a window approach is used, which assumes that the tag of a word primarily depends on its neighboring words. For each word, thus, a fixed-size window surrounding itself is assumed and the sub-sentence ranging within the window is considered. A standalone CNN is applied to this sub-sentence as explained earlier and predictions are attributed to the word in the center of the window. Following this approach, Poria et al. [52] employed a multi-level deep CNN to tag each word in a sentence as a possible aspect or non-aspect. Coupled with a set of linguistic patterns, their ensemble classifier managed to perform well in aspect detection.

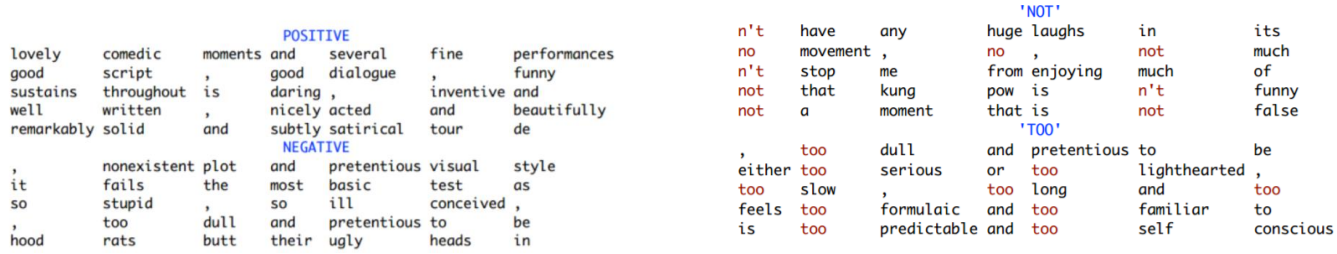
The ultimate goal of word-level classification is generally to assign a sequence of labels to the entire sentence. In such cases, structured prediction techniques such as conditional random field (CRF) are sometimes employed to better capture dependencies between adjacent class labels and finally generate cohesive label sequence giving maximum score to the whole sentence [53].

To get a larger contextual range, the classic window approach is often coupled with a time-delay neural network (TDNN) [54]. Here, convolutions are performed across all windows throughout the sequence. These convolutions are generally constrained by defining a kernel having a certain width. Thus, while the classic window approach only considers the words in the window around the word to be labeled, TDNN considers all windows of words in the sentence at the same time. At times, TDNN layers are also stacked like CNN architectures to extract local features in lower layers and global features in higher layers [5].

B. Applications

In this section, we present some of the crucial works that employed CNNs on NLP tasks to set state-of-the-art benchmarks in their respective times.

Kim [50] explored using the above architecture for a variety of sentence classification tasks, including sentiment, subjectivity and question type classification, showing competitive results. This work was quickly adapted by researchers given its simple yet effective network. After training for a specific task, the randomly initialized convolutional kernels became specific n-gram feature detectors that were useful for that target task (Fig. 7). This simple network, however, had many shortcomings with the CNN's inability to model long distance dependencies standing as the main issue.



(a) Figure A (b) Figure B

Fig. 7: Top 7-grams by four learned 7-gram kernels; each kernel is sensitive to a specific kind of 7-gram (Figure Source: Kalchbrenner et al. [49])

This issue was partly handled by Kalchbrenner et al. [49], who published a prominent paper where they proposed a dynamic convolutional neural network (DCNN) for semantic modeling of sentences. They proposed dynamic k-max pooling strategy which, given a sequence $\mathbf{p}$ selects the k most active features. The selection preserved the order of the features but was insensitive to their specific positions (Fig. 8). Built on the concept of TDNN, they added this dynamic k-max pooling strategy to create a sentence model. This combination allowed filters with small width to span across a long range within the input sentence, thus accumulating crucial information across the sentence. In the induced subgraph (Fig. 8), higher order features had highly variable ranges that could be either short and focused or global and long as the input sentence. They applied their model on multiple tasks, including sentiment prediction and question type classification, achieving significant results. Overall, this work commented on the range of individual kernels while trying to model contextual semantics and proposed a way to extend their reach.

Tasks involving sentiment analysis also require effective extraction of aspects along with their sentiment polarities [55]. Ruder et al. [56] applied a CNN where in the input they concatenated an aspect vector with the word embeddings to get competitive results. CNN modeling approach varies amongst different length of texts. Such differences were seen in many works like Johnson and Zhang [23], where performance on longer text worked well as opposed to shorter texts. Wang et al. [57] proposed the usage of CNN for modeling representations of short texts, which suffer from the lack of available context and, thus, require extra efforts to create meaningful representations. The authors proposed semantic clustering which introduced multi-scale semantic units to be used as external knowledge for the short texts. CNN was used to combine these units and form the overall representation. In fact, this requirement of high context information can be thought of as a caveat for CNN-based models. NLP tasks involving microtexts using CNN-based methods often require the need of additional information and external knowledge to perform as per expectations. This fact was also observed in [58], where authors performed sarcasm detection in Twitter texts using a CNN network. Auxiliary support, in the form of pre-trained networks trained on emotion, sentiment and personality datasets was used to achieve state-of-the-art performance.

CNNs have also been extensively used in other tasks. For example, Denil et al. [59] applied DCNN to map meanings of words that constitute a sentence to that of documents for summarization. The DCNN learned convolution filters at both the sentence and document level, hierarchically learning to capture and compose low-level lexical features into high-level semantic concepts. The focal point of this work was the introduction of a novel visualization technique of the learned representations, which provided insights not only in the learning process but also for automatic summarization of texts.

CNN models are also suitable for certain NLP tasks that require semantic matching beyond classification [60]. A similar model to the above CNN architecture (Fig. 6) was explored in [61] for information retrieval. The CNN was used for projecting queries and documents to a fixed-dimension semantic space, where cosine similarity between the query and documents was used for ranking documents regarding a specific query. The model attempted to extract rich contextual structures in a query or a document by considering a temporal context window in a word sequence. This captured the contextual features at the word n-gram level. The salient word n-grams is then discovered by the convolution and max-pooling layers which are then aggregated to form the overall sentence vector.

In the domain of QA, Yih et al. [62] proposed to measure the semantic similarity between a question and entries in a knowledge base (KB) to determine what supporting fact in the KB to look for when answering a question. To create semantic representations, a CNN similar to the one in Fig. 6 was used. Unlike the classification setting, the supervision signal came from positive or negative text pairs (e.g., query-document), instead of class labels. Subsequently, Dong et al. [63] introduced a multi-column CNN (MCCNN) to analyze and understand questions from multiple aspects and create their representations. MCCNN used multiple column networks to extract information from aspects comprising answer types and context from the input questions. By representing entities and relations in the KB with low-dimensional vectors, they used question-answer pairs to train the CNN model so as to rank candidate answers. Severyn and Moschitti [64] also used CNN network to model optimal representations of question and answer sentences. They proposed additional features in the embeddings in the form of relational information given by matching words between the question and answer pair. These parameters were tuned by the

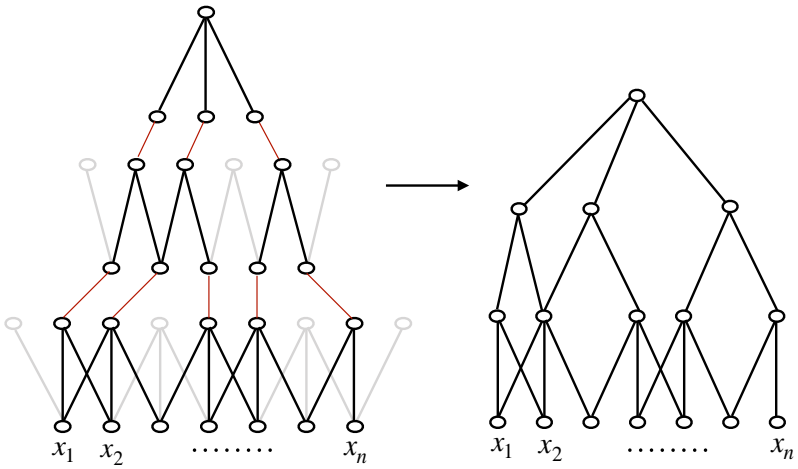


Fig. 8: DCNN subgraph. With dynamic pooling, a filter with small width at the higher layers can relate phrases far apart in the input sentence (Figure Source: Kalchbrenner et al. [49])

network. This simple network was able to produce comparable results to state-of-the-art methods.

CNNs are wired in a way to capture the most important information in a sentence. Traditional max-pooling strategies perform this in a translation invariant form. However, this often misses valuable information present in multiple facts within the sentence. To overcome this loss of information for multiple-event modeling, Chen et al. [65] proposed a modified pooling strategy: dynamic multi-pooling CNN (DMCNN). This strategy used a novel dynamic multi-pooling layer that, as the name suggests, incorporates event triggers and arguments to reserve more crucial information from the pooling layer.

CNNs inherently provide certain required features like local connectivity, weight sharing, and pooling. This puts forward some degree of invariance which is highly desired in many tasks. Speech recognition also requires such invariance and, thus, Abdel-Hamid et al. [66] used a hybrid CNN-HMM model which provided invariance to frequency shifts along the frequency axis. This variability is often found in speech signals due to speaker differences. They also performed limited weight sharing which led to a smaller number of pooling parameters, resulting in lower computational complexity. Palaz et al. [67] performed extensive analysis of CNN-based speech recognition systems when given raw speech as input. They showed the ability of CNNs to directly model the relationship between raw input and phones, creating a robust automatic speech recognition system.

Tasks like machine translation require perseverance of sequential information and long-term dependency. Thus, structurally they are not well suited for CNN networks, which lack these features. Nevertheless, Tu et al. [68] addressed this task by considering both the semantic similarity of the translation pair and their respective contexts. Although this method did not address the sequence perseverance problem, it allowed them to get competitive results amongst other benchmarks.

Overall, CNNs are extremely effective in mining semantic clues in contextual windows. However, they are very data heavy models. They include a large number of trainable parameters which require huge training data. This poses a problem when scarcity of data arises. Another persistent issue with CNNs is their inability to model long-distance contextual information and preserving sequential order in their representations [49, 68]. Other networks like recursive models (explained below) reveal themselves as better suited for such learning.

IV. Recurrent Neural Networks

RNNs [69] use the idea of processing sequential information. The term "recurrent" applies as they perform the same task over each instance of the sequence such that the output is dependent on the previous computations and results. Generally, a fixed-size vector is produced to represent a sequence by feeding tokens one by one to a recurrent unit. In a way, RNNs have "memory" over previous computations and use this information in current processing. This template is naturally suited for many NLP tasks such as language modeling [2, 70, 71], machine translation [72, 73, 74], speech recognition [75, 76, 77, 78], image captioning [79]. This made RNNs increasingly popular for NLP applications in recent years.

A. Need for Recurrent Networks

In this section, we analyze the fundamental properties that favored the popularization of RNNs in a multitude of NLP tasks. Given that an RNN performs sequential processing by modeling units in sequence, it has the ability to capture the inherent sequential nature present in language, where units are characters, words or even sentences. Words in a language develop their semantical meaning based on the previous words in the sentence. A simple example stating this would be the difference in meaning between "dog" and "hot dog". RNNs are tailor-made for modeling such context dependencies in language and similar sequence modeling tasks, which resulted to be a strong motivation for researchers to use RNNs over CNNs in these areas.

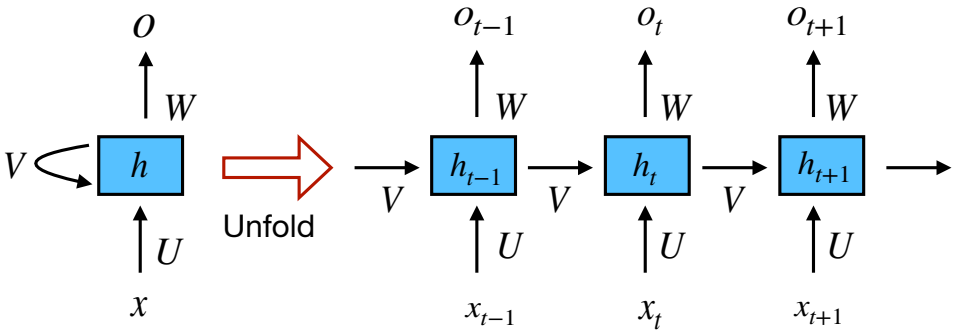


Fig. 9: Simple RNN network (Figure Source: LeCun et al. [90])

Another factor aiding RNN's suitability for sequence modeling tasks lies in its ability to model variable length of text, including very long sentences, paragraphs and even documents [80]. Unlike CNNs, RNNs have flexible computational steps that provide better modeling capability and create the possibility to capture unbounded context. This ability to handle input of arbitrary length became one of the selling points of major works using RNNs [81].

Many NLP tasks require semantic modeling over the whole sentence. This involves creating a gist of the sentence in a fixed dimensional hyperspace. RNN's ability to summarize sentences led to their increased usage for tasks like machine translation [82] where the whole sentence is summarized to a fixed vector and then mapped back to the variable-length target sequence.

RNN also provides the network support to perform time distributed joint processing. Most of the sequence labeling tasks like POS tagging [32] come under this domain. More specific use cases include applications such as multi-label text categorization [83], multimodal sentiment analysis [84, 85, 86], and subjectivity detection [87].

The above points enlist some of the focal reasons that motivated researchers to opt for RNNs. However, it would be gravely wrong to make conclusions on the superiority of RNNs over other deep networks. Recently, several works provided contrasting evidence on the superiority of CNNs over RNNs. Even in RNN-suited tasks like language modeling, CNNs achieved competitive performance over RNNs [88]. Both CNNs and RNNs have different objectives when modeling a sentence. While RNNs try to create a composition of an arbitrarily long sentence along with unbounded context, CNNs try to extract the most important n-grams. Although they prove an effective way to capture n-gram features, which is approximately sufficient in certain sentence classification tasks, their sensitivity to word order is restricted locally and long-term dependencies are typically ignored.

Yin et al. [89] provided interesting insights on the comparative performance between RNNs and CNNs. After testing on multiple NLP tasks that included sentiment classification, QA, and POS tagging, they concluded that there is no clear winner: the performance of each network depends on the global semantics required by the task itself.

Below, we discuss some of the RNN models extensively used in the literature.

B. RNN models

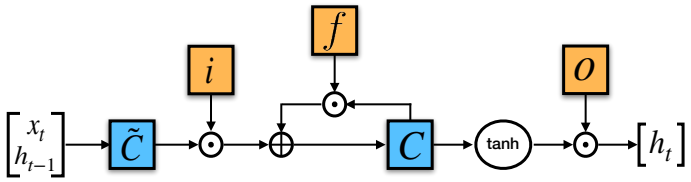
1) **Simple RNN***:* In the context of NLP, RNNs are primarily based on Elman network [69] and they are originally three-layer networks. Fig. 9 illustrates a more general RNN which is unfolded across time to accommodate a whole sequence. In the figure, x_t is taken as the input to the network at time step t and s_t represents the hidden state at the same time step. Calculation of s_t is based as per the equation:

$$\mathbf{s_t} = f(U\mathbf{x_t} + W\mathbf{s_{t-1}}) \tag{11}$$

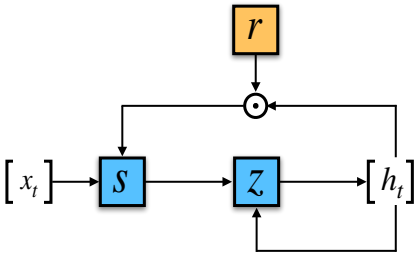
Thus, s_t is calculated based on the current input and the previous time step's hidden state. The function f is taken to be a non-linear transformation such as $tanh, ReLU$ and U, V, W account for weights that are shared across time. In the context of NLP, x_t typically comprises of one-hot encodings or embeddings. At times, they can also be abstract representations of textual content. o_t illustrates the output of the network which is also often subjected to non-linearity, especially when the network contains further layers downstream.

The hidden state of the RNN is typically considered to be its most crucial element. As stated before, it can be considered as the network's memory element that accumulates information from other time steps. In practice, however, these simple RNN networks suffer from the infamous *vanishing gradient* problem, which makes it really hard to learn and tune the parameters of the earlier layers in the network.

This limitation was overcome by various networks such as long short-term memory (LSTM), gated recurrent units (GRUs), and residual networks (ResNets), where the first two are the most used RNN variants in NLP applications.



(1) Long Short-Term Memory



(2) Gated Recurrent Unit

Fig. 10: Illustration of an LSTM and GRU gate (Figure Source: Chung et al. [81])

2) **Long Short-Term Memory***:* LSTM [91, 92] (Fig. 10) has additional "forget" gates over the simple RNN. Its unique mechanism enables it to overcome both the *vanishing* and *exploding* gradient problem.

Unlike the vanilla RNN, LSTM allows the error to back-propagate through unlimited number of time steps. Consisting of three gates: input, forget and output gates, it calculates the hidden state by taking a combination of these three gates as per the equations below:

$$\mathbf{x} = \begin{bmatrix} \boldsymbol{h}_{t-1} \\ \boldsymbol{x}_t \end{bmatrix} \tag{12}$$

$$f_t = \sigma(W_f.\mathbf{x} + b_f) \tag{13}$$

$$i_t = \sigma(W_i.\mathbf{x} + b_i) \tag{14}$$

$$o_t = \sigma(W_o.\mathbf{x} + b_o) \tag{15}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot tanh(W_c.X + b_c) \tag{16}$$

$$h_t = o_t \odot tanh(c_t) \tag{17}$$

3) **Gated Recurrent Units***:* Another gated RNN variant called GRU [82] (Fig. 10) of lesser complexity was invented with empirically similar performances to LSTM in most tasks. GRU comprises of two gates, reset gate and update gate, and handles the flow of information like an LSTM sans a memory unit. Thus, it exposes the whole hidden content without any control. Being less complex, GRU can be a more efficient RNN than LSTM. The working of GRU is as follows:

$$\mathbf{z} = \sigma(U_z.\mathbf{x_t} + W_z.\mathbf{h_{t-1}}) \tag{18}$$

$$\mathbf{r} = \sigma(U_r.\mathbf{x_t} + W_r.\mathbf{h_{t-1}}) \tag{19}$$

$$\mathbf{s_t} = tanh(U_z.\mathbf{x_t} + W_s.(h_{t-1} \odot \mathbf{r})) \tag{20}$$

$$\mathbf{h_t} = (1 - \mathbf{z}) \odot \mathbf{s_t} + \mathbf{z} \odot \mathbf{h_{t-1}} \tag{21}$$

Researchers often face the dilemma of choosing the appropriate gated RNN. This also extends to developers working in NLP. Throughout the history, most of the choices over the RNN variant tended to be heuristic. Chung et al. [81] did a critical comparative evaluation of the three RNN variants mentioned above, although not on NLP tasks. They evaluated their work on tasks relating to polyphonic music modeling and speech signal modeling. Their evaluation clearly demonstrated the superiority of the gated units (LSTM and GRU) over the traditional simple RNN (in their case, using $tanh$ activation) (Fig. 11). However, they could not make any concrete conclusion about which of the two gating units was better. This fact has been noted in other works too and, thus, people often leverage on other factors like computing power while choosing between the two.

C. Applications

1) RNN for word-level classification: RNNs have had a huge presence in the field of word-level classification. Many of their applications stand as state of the art in their respective tasks. Lample et al. [93] proposed to use bidirectional LSTM

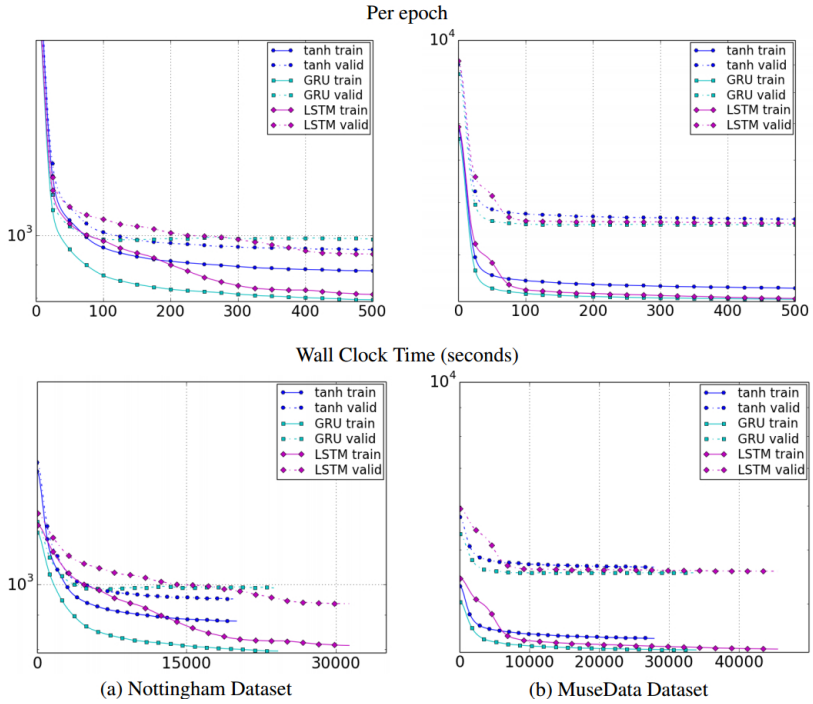


Fig. 11: Learning curves for training and validation sets of different types of units with respect to (top) the number of iterations and (bottom) the wall clock time. y-axis corresponds to the negative log likelihood of the model shown in log-scale (Figure source: Chung et al. [81])

for NER. The network captured arbitrarily long context information around the target word (curbing the limitation of a fixed window size) resulting in two fixed-size vector, on top of which another fully-connected layer was built. They used a CRF layer at last for the final entity tagging.

RNNs have also shown considerable improvement in language modeling over traditional methods based on count statistics. Pioneering work in this field was done by Graves [94], who introduced the effectiveness of RNNs in modeling complex sequences with long range context structures. He also proposed deep RNNs where multiple layers of hidden states were used to enhance the modeling. This work established the usage of RNNs on tasks beyond the context of NLP. Later, Sundermeyer et al. [95] compared the gain obtained by replacing a feed-forward neural network with an RNN when conditioning the prediction of a word on the words ahead. In their work, they proposed a typical hierarchy in neural network architectures where feed-forward neural networks gave considerable improvement over traditional count-based language models, which in turn were superseded by RNNs and later by LSTMs. An important point that they mentioned was the applicability of their conclusions to a variety of other tasks such as statistical machine translation [96].

2) RNN for sentence-level classification: Wang et al. [25] proposed encoding entire tweets with LSTM, whose hidden state is used for predicting sentiment polarity. This simple strategy proved competitive to the more complex DCNN structure by Kalchbrenner et al. [49] designed to endow CNN models with ability to capture long-term dependencies. In a special case studying negation phrase, the authors also showed that the dynamics of LSTM gates can capture the reversal effect of the word *not*.

Similar to CNN, the hidden state of an RNN can also be used for semantic matching between texts. In dialogue systems, Lowe et al. [97] proposed to match a message with candidate responses with Dual-LSTM, which encodes both as fixed-size vectors and then measure their inner product as the basis to rank candidate responses.

3) RNN for generating language: A challenging task in NLP is generating natural language, which is another natural application of RNNs. Conditioned on textual or visual data, deep LSTMs have been shown to generate reasonable task-specific text in tasks such as machine translation, image captioning, etc. In such cases, the RNN is termed a decoder.

In [74], the authors proposed a general deep LSTM encoder-decoder framework that maps a sequence to another sequence. One LSTM is used to encode the "source" sequence as a fixed-size vector, which can be text in the original language (machine translation), the question to be answered (QA) or the message to be replied to (dialogue systems). The vector is used as the initial state of another LSTM, named the decoder. During inference, the decoder generates tokens one by one, while updating its hidden state with the last generated token. Beam search is often used to approximate the optimal sequence.

Sutskever et al. [74] experimented with 4-layer LSTM on a machine translation task in an end-to-end fashion, showing competitive results. In [99], the same encoder-decoder framework is employed to model human conversations. When trained on more than 100 million message-response pairs, the LSTM decoder is able to generate very interesting responses in the open

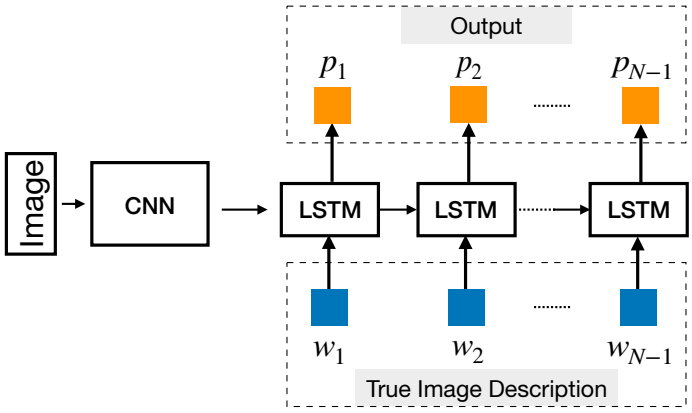


Fig. 12: LSTM decoder combined with a CNN image embedder to generate image captioning (Figure source: Vinyals et al. [98])

domain. It is also common to condition the LSTM decoder on additional signal to achieve certain effects. In [100], the authors proposed to condition the decoder on a constant persona vector that captures the personal information of an individual speaker. In the above cases, language is generated based mainly on the semantic vector representing textual input. Similar frameworks have also been successfully used in image-based language generation, where visual features are used to condition the LSTM decoder (Fig. 12).

Visual QA is another task that requires language generation based on both textual and visual clues. Malinowski et al. [101] were the first to provide an end-to-end deep learning solution where they predicted the answer as a set of words conditioned on the input image modeled by a CNN and text modeled by an LSTM (Fig. 13).

Kumar et al. [102] tackled this problem by proposing an elaborated network termed dynamic memory network (DMN), which had four sub-modules. The idea was to repeatedly attend to the input text and image to form episodes of information improved at each iteration. Attention networks were used for fine-grained focus on input text phrases.

D. Attention Mechanism

One potential problem that the traditional encoder-decoder framework faces is that the encoder at times is forced to encode information which might not be fully relevant to the task at hand. The problem arises also if the input is long or very information-rich and selective encoding is not possible.

For example, the task of text summarization can be cast as a sequence-to-sequence learning problem, where the input is the original text and the output is the condensed version. Intuitively, it is unrealistic to expect a fixed-size vector to encode all information in a piece of text whose length can potentially be very long. Similar problems have also been reported in machine translation [103].

In tasks such as text summarization and machine translation, certain alignment exists between the input text and the output text, which means that each token generation step is highly related to a certain part of the input text. This intuition inspires the

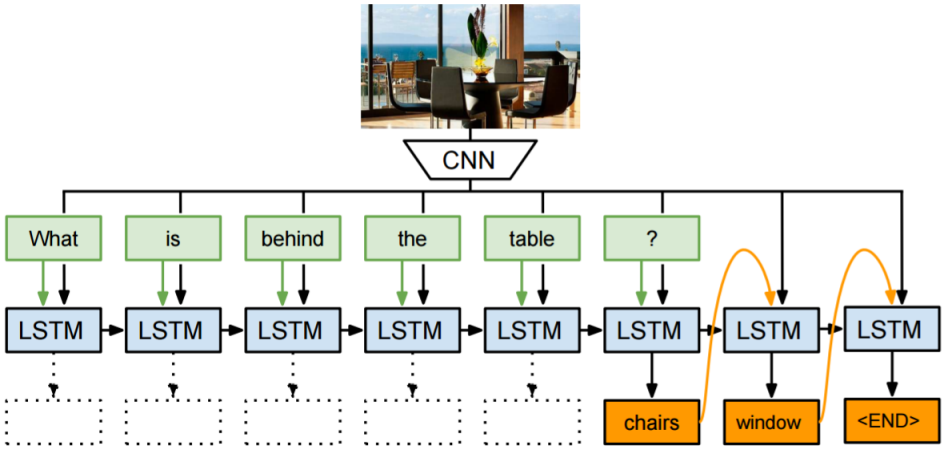


Fig. 13: Neural-image QA (Figure source: Malinowski et al. [101])

attention mechanism. This mechanism attempts to ease the above problems by allowing the decoder to refer back to the input sequence. Specifically during decoding, in addition to the last hidden state and generated token, the decoder is also conditioned on a "context" vector calculated based on the input hidden state sequence. The attention mechanism can be broadly seen as mapping a query and a set of key-value pairs to an output, where all the mentioned components are vectors. The output is a combination of the values whose weights are determined by the compatibility between the query and the corresponding keys. This output amounts to the "context" of the input used in decoding the output.

Bahdanau et al. [103] first applied the attention mechanism to machine translation, which improved the performance especially for long sequences. In their work, the attention signal over the input hidden state sequence is determined with a multi-layer perceptron by the last hidden state of the decoder. By visualizing the attention signal over the input sequence during each decoding step, a clear alignment between the source and target language can be demonstrated (Fig. 14).

A similar approach was applied to the task of summarization by Rush et al. [104] where each output word in the summary was conditioned on the input sentence through an attention mechanism. The authors performed abstractive summarization which is not very conventional as opposed to extractive summarization, but can be scaled up to large data with minimal linguistic input.

In image captioning, Xu et al. [105] conditioned the LSTM decoder on different parts of the input image during each decoding step. Attention signal was determined by the previous hidden state and CNN features. In [106], the authors casted the syntactical parsing problem as a sequence-to-sequence learning task by linearizing the parsing tree. The attention mechanism proved to be more data-efficient in this work. A further step in referring to the input sequence was to directly copy words or sub-sequences of the input onto the output sequence under a certain condition [107], which was useful in tasks such as dialogue generation and text summarization. Copying or generation was chosen at each time step during decoding [108].

In aspect-based sentiment analysis, Wang et al. [109] proposed an attention-based solution where they used aspect embeddings to provide additional support during classification (Fig. 15). The attention module focused on selective regions of the sentence which affected the aspect to be classified. This can be seen in Fig. 17 where, for the aspect *service* in (a), the attention module dynamically focused on the phrase "fastest delivery times" and in (b) with the aspect *food*, it identified multiple key-points across the sentence that included "tasteless" and "too sweet". Recently, Ma et al. [110] augmented LSTM with a hierarchical attention mechanism consisting of a target-level attention and a sentence-level attention to exploit commonsense knowledge for targeted aspect-based sentiment analysis.

On the other hand, Tang et al. [111] adopted a solution based on a memory network (also known as MemNet [112]), which employed multiple-hop attention. The multiple attention computation layer on the memory led to improved lookup for most informational regions in the memory and subsequently aided the classification. Their work stands as the state of the art in this domain.

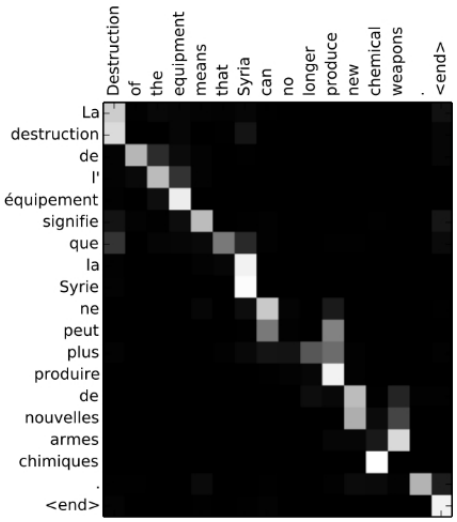


Fig. 14: Word alignment matrix (Figure source: Bahdanau et al. [103])

Given the intuitive applicability of attention modules, they are still being actively investigated by NLP researchers and adopted for an increasing number of applications.

E. Parallelized Attention: The Transformer

Both CNNs and RNNs have been crucial in sequence transduction applications involving the encoder-decoder architecture. Attention-based mechanisms, as described above, have further boosted the capabilities of these models. However, one of the bottlenecks suffered by these architectures is the sequential processing at the encoding step. To address this, Vaswani et al. [113] proposed the *Transformer* which dispensed the recurrence and convolutions involved in the encoding step entirely and based models only on attention mechanisms to capture the global relations between input and output. As a result, the overall architecture became more parallelizable and required lesser time to train along with positive results on tasks ranging from translation to parsing.

The Transformer consists stacked layers in both encoder and decoder components. Each layer has two sub-layers comprising *multi-head attention layer* (Figure 17) followed by a position-wise feed forward network. For set of queries Q, keys K and

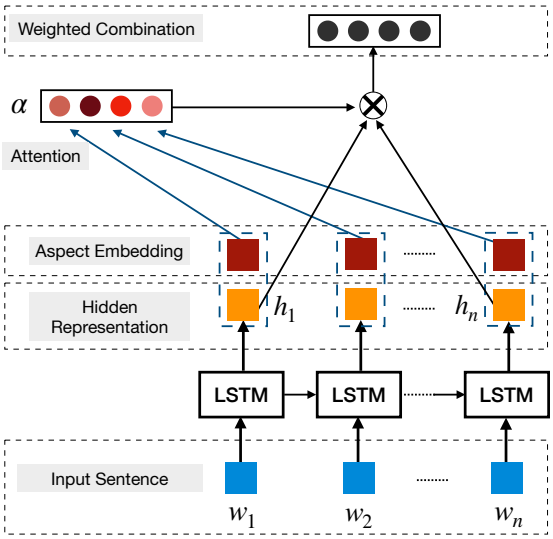


Fig. 15: Aspect classification using attention (Figure source: Wang et al. [109])

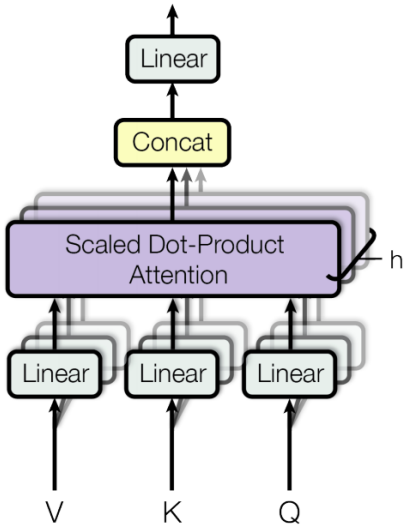


Fig. 16: Multi-head Attention: Vaswani et al. [113])

values V, the multi-head attention module performs attention h times where the computation can be seen as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W^o \tag{22}$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{23}$$

$$\text{and Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{24}$$

here, $W_i^{[.]}$ and W^o are projection parameters. Incorporating other techniques such as residual connections [114], layer normalization [115], dropouts, positional encodings, and others, the model achieves state-of-the-art results in English-German and English-French translation and constituency parsing.

V. Recursive Neural Networks

Recurrent neural networks represent a natural way to model sequences. Arguably, however, language exhibits a natural recursive structure, where words and sub-phrases combine into phrases in a hierarchical manner. Such structure can be represented by a constituency parsing tree. Thus, tree-structured models have been used to better make use of such syntactic interpretations of sentence structure [4]. Specifically, in a recursive neural network, the representation of each non-terminal node in a parsing tree is determined by the representations of all its children.

A. Basic model

In this section, we describe the basic structure of recursive neural networks. As shown in Fig. 18a and 18b, the network g defines a compositional function on the representations of phrases or words (b, c or a, p_1) to compute the representation of a higher-level phrase (p_1 or p_2). The representations of all nodes take the same form.

Socher et al. [4] described multiple variations of this model. In its simplest form, g is defined as:

$$p_1 = tanh\left(W \begin{bmatrix} b \\ c \end{bmatrix}\right), p_2 = tanh\left(W \begin{bmatrix} a \\ p_1 \end{bmatrix}\right) \tag{25}$$

in which the representation for each node is a d-dimensional vector and $W \in \mathcal{R}^{D \times 2D}$.

Another variation is the MV-RNN [116]. The idea is to represent every word and phrase as both a matrix and a vector. When two constituents are combined, the matrix of one is multiplied with the vector of the other:

$$p_1 = tanh\left(W \begin{bmatrix} Cb \\ Bc \end{bmatrix}\right), P_1 = tanh\left(W_M \begin{bmatrix} B \\ C \end{bmatrix}\right) \tag{26}$$

in which $b, c, p_1 \in \mathcal{R}^D$, $B, C, P_1 \in \mathcal{R}^{D \times D}$, and $W_M \in \mathcal{R}^{D \times 2D}$. Compared to the vanilla form, MV-RNN parameterizes the compositional function with matrices corresponding to the constituents.

The recursive neural tensor network (RNTN) is proposed to introduce more interaction between the input vectors without making the number of parameters exceptionally large like MV-RNN. RNTN is defined by:

$$p_1 = tanh\left(\begin{bmatrix} b \\ c \end{bmatrix}^T V^{[1:D]} \begin{bmatrix} b \\ c \end{bmatrix} + W \begin{bmatrix} b \\ c \end{bmatrix}\right) \tag{27}$$

where $V \in \mathcal{R}^{2D \times 2D \times D}$ is a tensor that defines multiple bilinear forms.

B. Applications

One natural application of recursive neural networks is parsing [10]. A scoring function is defined on the phrase representation to calculate the plausibility of that phrase. Beam search is usually applied for searching the best tree. The model is trained with the max-margin objective [117].

Based on recursive neural networks and the parsing tree, Socher et al. [4] proposed a phrase-level sentiment analysis framework (Fig. 19), where each node in the parsing tree can be assigned a sentiment label.

Socher et al. [116] classified semantic relationships such as cause-effect or topic-message between nominals in a sentence by building a single compositional semantics for the minimal constituent including both terms. Bowman et al. [118] proposed to classify the logical relationship between sentences with recursive neural networks. The representations for both sentences are fed to another neural network for relationship classification. They show that both vanilla and tensor versions of the recursive unit performed competitively in a textual entailment dataset.

To avoid the gradient vanishing problem, LSTM units have also been applied to tree structures in [119]. The authors showed improved sentence representation over linear LSTM models, as clear improvement in sentiment analysis and sentence relatedness test was observed.

VI. Deep reinforced models and deep unsupervised learning

A. Reinforcement learning for sequence generation

Reinforcement learning is a method of training an agent to perform discrete actions before obtaining a reward. In NLP, tasks concerning language generation can sometimes be cast as reinforcement learning problems.

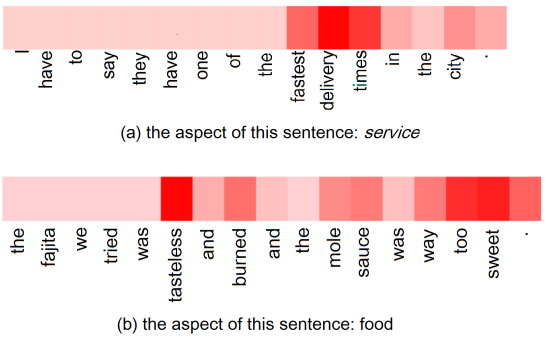
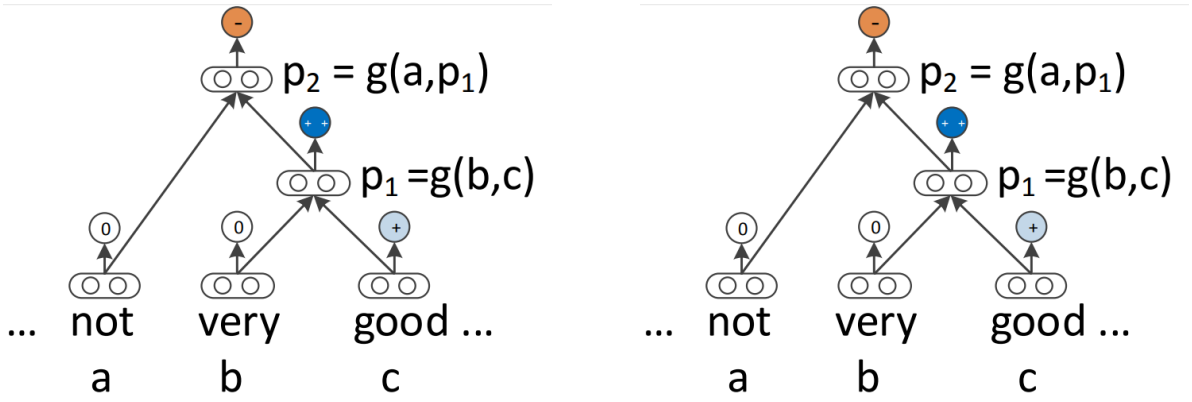


Fig. 17: Focus of attention module on the sentence for certain aspects (Figure source: Wang et al. [109])



(a) Recursive neural networks for phrase-level sentiment classification (Figure source: Socher et al. [4])

(b) Recursive neural networks iteratively form high-level representation from lower-level representations.

Fig. 18: Recursive Neural Networks

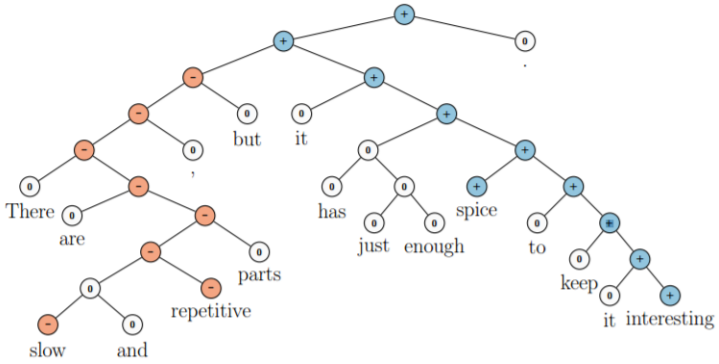


Fig. 19: Recursive neural networks applied on a sentence for sentiment classification. Note that "but" plays a crucial role on determining the sentiment of the whole sentence (Figure source: Socher et al. [4])

In its original formulation, RNN language generators are typically trained by maximizing the likelihood of each token in the ground-truth sequence given the current hidden state and the previous tokens. Termed "teacher forcing", this training scheme provides the real sequence prefix to the generator during each generation (loss evaluation) step. At test time, however, ground-truth tokens are then replaced by a token generated by the model itself. This discrepancy between training and inference, termed "exposure bias" [120, 121], can yield errors that can accumulate quickly along the generated sequence.

Another problem with the word-level maximum likelihood strategy, when training auto-regressive language generation models, is that the training objective is different from the test metric. It is unclear how the n-gram overlap based metrics (BLEU, ROUGE) used to evaluate these tasks (machine translation, dialogue systems, etc.) can be optimized with the word-level training strategy. Empirically, dialogue systems trained with word-level maximum likelihood also tend to produce dull and short-sighted responses [122], while text summarization tends to produce incoherent or repetitive summaries [108].

Reinforcement learning offers a prospective to solve the above problems to a certain extent. In order to optimize the non-differentiable evaluation metrics directly, Ranzato et al. [121] applied the REINFORCE algorithm [123] to train RNN-based models for several sequence generation tasks (e.g., text summarization, machine translation and image captioning), leading to improvements compared to previous supervised learning methods. In such a framework, the generative model (RNN) is viewed as an agent, which interacts with the external environment (the words and the context vector it sees as input at every time step). The parameters of this agent defines a policy, whose execution results in the agent picking an action, which refers to predicting the next word in the sequence at each time step. After taking an action the agent updates its internal state (the hidden units of RNN). Once the agent has reached the end of a sequence, it observes a reward. This reward can be any developer-defined metric tailored to a specific task. For example, Li et al. [122] defined 3 rewards for a generated sentence based on ease of answering, information flow, and semantic coherence.

There are two well-known shortcomings of reinforcement learning. To make reinforcement learning tractable, it is desired to carefully handle the state and action space [124, 125], which in the end may restrict expressive power and learning capacity of the model. Secondly, the need for training the reward functions makes such models hard to design and measure at run time [126, 127].

Another approach for sequence-level supervision is to use the adversarial training technique [128], where the training objective

for the language generator is to fool another discriminator trained to distinguish generated sequences from real sequences. The generator G and the discriminator D are trained jointly in a min-max game which ideally leads to G, generating sequences indistinguishable from real ones. This approach can be seen as a variation of generative adversarial networks in [128], where G and D are conditioned on certain stimuli (for example, the source image in the task of image captioning). In practice, the above scheme can be realized under the reinforcement learning paradigm with policy gradient. For dialogue systems, the discriminator is analogous to a human Turing tester, who discriminates between human and machine-produced dialogues [129].

B. Unsupervised sentence representation learning

Similar to word embeddings, distributed representation for sentences can also be learned in an unsupervised fashion. The result of such unsupervised learning are "sentence encoders", which map arbitrary sentences to fixed-size vectors that can capture their semantic and syntactic properties. Usually an auxiliary task has to be defined for the learning process.

Similar to the skip-gram model [8] for learning word embeddings, the skip-thought model [130] was proposed for learning sentence representation, where the auxiliary task was to predict two adjacent sentences (before and after) based on the given sentence. The seq2seq model was employed for this learning task. One LSTM encoded the sentence to a vector (distributed representation). Two other LSTMs decoded such representation to generate the target sequences. Standard seq2seq training process was used. After training, the encoder could be seen as a generic feature extractor (word embeddings were also learned in the same time).

Kiros et al. [130] verified the quality of the learned sentence encoder on a range of sentence classification tasks, showing competitive results with a simple linear model based on the static feature vectors. However, the sentence encoder can also be fine-tuned in the supervised learning task as part of the classifier. Dai and Le [43] investigated the use of the decoder to reconstruct the encoded sentence itself, which resembled an autoencoder [131].

Language modeling could also be used as an auxiliary task when training LSTM encoders, where the supervision signal came from the prediction of the next token. Dai and Le [43] conducted experiments on initializing LSTM models with learned parameters on a variety of tasks. They showed that pre-training the sentence encoder on a large unsupervised corpus yielded better accuracy than only pre-training word embeddings. Also, predicting the next token turned out to be a worse auxiliary objective than reconstructing the sentence itself, as the LSTM hidden state was only responsible for a rather short-term objective.

C. Deep generative models

Recent success in generating realistic images has driven a series of efforts on applying deep generative models to text data. The promise of such research is to discover rich structure in natural language while generating realistic sentences from a latent code space. In this section, we review recent research on achieving this goal with variational autoencoders (VAEs) [132] and generative adversarial networks (GANs) [128].

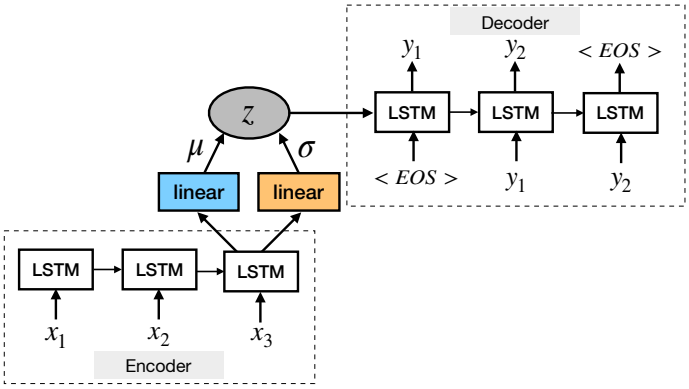


Fig. 20: RNN-based VAE for sentence generation (Figure source: Bowman et al. [133])

Standard sentence autoencoders, as in the last section, do not impose any constraint on the latent space, as a result, they fail when generating realistic sentences from arbitrary latent representations [133]. The representations of these sentences may often occupy a small region in the hidden space and most of regions in the hidden space do not necessarily map to a realistic sentence [134]. They cannot be used to assign probabilities to sentences or to sample novel sentences [133].

The VAE imposes a prior distribution on the hidden code space which makes it possible to draw proper samples from the model. It modifies the autoencoder architecture by replacing the deterministic encoder function with a learned posterior recognition model. The model consists of encoder and generator networks which encode data examples to latent representation and generate samples from the latent space, respectively. It is trained by maximizing a variational lower bound on the log-likelihood of observed data under the generative model.

Story (1: 1 supporting fact)	Support	Hop 1	Hop 2	Hop 3
Daniel went to the bathroom.		0.00	0.00	0.03
Mary travelled to the hallway.		0.00	0.00	0.00
John went to the bedroom.		0.37	0.02	0.00
John travelled to the bathroom.	yes	0.60	0.98	0.96
Mary went to the office.		0.01	0.00	0.00
Where is John? Answer: bathroom	**Prediction: bathroom**			

Story (16: basic induction)	Support	Hop 1	Hop 2	Hop 3
Brian is a frog.	yes	0.00	0.98	0.00
Lily is gray.		0.07	0.00	0.00
Brian is yellow.	yes	0.07	0.00	1.00
Julius is green.		0.06	0.00	0.00
Greg is a frog.	yes	0.76	0.02	0.00
What color is Greg? Answer: yellow	**Prediction: yellow**			

Fig. 21: Multiple supporting facts were retrieved from the memory in order to answer a specific question using an attention mechanism. The first hop uncovered the need for additional hops (Figure source: Sukhbaatar et al. [138])

Bowman et al. [133] proposed an RNN-based variational autoencoder generative model that incorporated distributed latent representations of entire sentences (Fig. 20). Unlike vanilla RNN language models, this model worked from an explicit global sentence representation. Samples from the prior over these sentence representations produced diverse and well-formed sentences.

Hu et al. [135] proposed generating sentences whose attributes are controlled by learning disentangled latent representations with designated semantics. The authors augmented the latent code in the VAE with a set of structured variables, each targeting a salient and independent semantic feature of sentences. The model incorporated VAE and attribute discriminators, in which the VAE component trained the generator to reconstruct real sentences for generating plausible text, while the discriminators forced the generator to produce attributes coherent with the structured code. When trained on a large number of unsupervised sentences and a small number of labeled sentences, Hu et al. [135] showed that the model was able to generate plausible sentences conditioned on two major attributes of English: tense and sentiment.

GAN is another class of generative model composed of two competing networks. A generative neural network decodes latent representation to a data instance, while the discriminative network is simultaneously taught to discriminate between instances from the true data distribution and synthesized instances produced by the generator. GAN does not explicitly represent the true data distribution $p(x)$.

Zhang et al. [134] proposed a framework for employing LSTM and CNN for adversarial training to generate realistic text. The latent code z was fed to the LSTM generator at every time step. CNN acted as a binary sentence classifier which discriminated between real data and generated samples. One problem with applying GAN to text is that the gradients from the discriminator cannot properly back-propagate through discrete variables. In [134], this problem was solved by making the word prediction at every time "soft" at the word embedding space. Yu et al. [136] proposed to bypass this problem by modeling the generator as a stochastic policy. The reward signal came from the GAN discriminator judged on a complete sequence, and was passed back to the intermediate state-action steps using Monte Carlo search.

The evaluation of deep generative models has been challenging. For text, it is possible to create oracle training data from a fixed set of grammars and then evaluate generative models based on whether (or how well) the generated samples agree with the predefined grammar [137]. Another strategy is to evaluate BLEU scores of samples on a large amount of unseen test data. The ability to generate similar sentences to unseen real data is considered a measurement of quality [136].

VII. Memory-augmented Networks

The attention mechanism stores a series of hidden vectors of the encoder, which the decoder is allowed to access during the generation of each token. Here, the hidden vectors of the encoder can be seen as entries of the model's "internal memory". Recently, there has been a surge of interest in coupling neural networks with a form of memory, which the model can interact with.

In [112], the authors proposed memory networks for QA tasks. In synthetic QA, a series of statements (memory entries) were provided to the model as potential supporting facts to the question. The model learned to retrieve one entry at a time from memory based on the question and previously retrieved memory. In large-scale realistic QA, a large set of commonsense knowledge in the form of (subject, relation, object) triples were used as memory. Sukhbaatar et al. [138] extended this work and proposed end-to-end memory networks, where memory entries were retrieved in a "soft" manner with attention mechanism, thus enabling end-to-end training. Multiple rounds (hops) of information retrieval from memory were shown to be essential to good performance and the model was able to retrieve and reason about several supporting facts to answer a specific question (Fig. 21). Sukhbaatar et al. [138] also showed a special use of the model for language modeling, where each word in the sentence was seen as a memory entry. With multiple hops, the model yielded results comparable to deep LSTM models.

Furthermore, dynamic memory networks (DMN) [102] improved upon previous memory-based models by employing neural network models for input representation, attention, and answer mechanisms. The resulting model was applicable to a wide

range of NLP tasks (QA, POS tagging, and sentiment analysis), as every task could be cast to the <memory, question, answer> triple format. Xiong et al. [139] applied the same model to visual QA and proved that the memory module was applicable to visual signals.

VIII. Performance of different models on different NLP tasks

We summarize the performance of a series of deep learning methods on standard datasets developed in recent years on some major NLP topics. Our goal is to show the readers common datasets used in the community and state-of-the-art results with different models.

A. POS tagging

The WSJ-PTB (the Wall Street Journal part of the Penn Treebank Dataset) corpus contains 1.17 million tokens and has been widely used for developing and evaluating POS tagging systems. Giménez and Marquez [140] employed one-against-all SVM based on manually-defined features within a seven-word window, in which some basic n-gram patterns were evaluated to form binary features such as: "previous word is *the*", "two preceding tags are DT NN", etc. One characteristic of the POS tagging problem was the strong dependency between adjacent tags. With a simple left-to-right tagging scheme, this method modeled dependencies between adjacent tags only by feature engineering. In an effort to reduce feature engineering, Collobert et al. [5] relied on only word embeddings within the word window by a multi-layer perceptron. Incorporating CRF was proven useful in [5]. Santos and Zadrozny [32] concatenated word embeddings with character embeddings to better exploit morphological clues. In [32], the authors did not consider CRF, but since word-level decision was made on a context window, it can be seen that dependencies were modeled implicitly. Huang et al. [141] concatenated word embeddings and manually-designed word-level features and employed bidirectional LSTM to model arbitrarily long context. A series of ablative analysis suggested that bi-directionality and CRF both boosted performance. Andor et al. [142] showed a transition-based approach that produces competitive result with a simple feed-forward neural network. When applied to sequence tagging tasks, DMNs [102] essentially allowed for attending over the context multiple times by treating each RNN hidden state as a memory entry, each time focusing on different parts of the context.

TABLE II: **POS tagging**

Paper	Model	WSJ-PTB (per-token accuracy %)
Giménez and Marquez [140]	SVM with manual feature pattern	97.16
Collobert et al. [5]	MLP with word embeddings + CRF	97.29
Santos and Zadrozny [32]	MLP with character+word embeddings	97.32
Huang et al. [141]	LSTM	97.29
Huang et al. [141]	Bidirectional LSTM	97.40
Huang et al. [141]	LSTM-CRF	97.54
Huang et al. [141]	Bidirectional LSTM-CRF	97.55
Andor et al. [142]	Transition-based neural network	97.45
Kumar et al. [102]	DMN	97.56

B. Parsing

There are two types of parsing: dependency parsing, which connects individual words with their relations, and constituency parsing, which iteratively breaks text into sub-phrases. Transition-based methods are a popular choice since they are linear in the length of the sentence. The parser makes a series of decisions that read words sequentially from a buffer and combine them incrementally into the syntactic structure [143]. At each time step, the decision is made based on a stack containing available tree nodes, a buffer containing unread words and the obtained set of dependency arcs. Chen and Manning [143] modeled the decision making at each time step with a neural network with one hidden layer. The input layer contained embeddings of certain words, POS tags and arc labels, which came from the stack, the buffer and the set of arc labels.

Tu et al. [68] extended the work of Chen and Manning [143] by employing a deeper model with 2 hidden layers. However, both Tu et al. [68] and Chen and Manning [143] relied on manual feature selecting from the parser state, and they only took into account a limited number of latest tokens. Dyer et al. [144] proposed stack-LSTMs to model arbitrarily long history. The end pointer of the stack changed position as the stack of tree nodes could be pushed and popped. Zhou et al. [145] integrated beam search and contrastive learning for better optimization.

Transition-based models were applied to constituency parsing as well. Zhu et al. [146] based each transition action on features such as the POS tags and constituent labels of the top few words of the stack and the buffer. By uniquely representing the parsing tree with a linear sequence of labels, Vinyals et al. [106] applied the seq2seq learning method to this problem.

TABLE III: **Parsing** (UAS/LAS = Unlabeled/labeled Attachment Score; WSJ = The Wall Street Journal Section of Penn Treebank)

Parsing type	Paper	Model	WSJ
Dependency Parsing	Chen and Manning [143]	Fully-connected NN with features including POS	91.8/89.6 (UAS/LAS)
	Weiss et al. [147]	Deep fully-connected NN with features including POS	94.3/92.4 (UAS/LAS)
	Dyer et al. [144]	Stack-LSTM	93.1/90.9 (UAS/LAS)
	Zhou et al. [145]	Beam contrastive model	93.31/92.37 (UAS/LAS)
Constituency Parsing	Petrov et al. [148]	Probabilistic context-free grammars (PCFG)	91.8 (F1 Score)
	Socher et al. [10]	Recursive neural networks	90.29 (F1 Score)
	Zhu et al. [146]	Feature-based transition parsing	91.3 (F1 Score)
	Vinyals et al. [106]	seq2seq learning with LSTM+Attention	93.5 (F1 Score)

TABLE IV: **Named-Entity Recognition**

Paper	Model	CoNLL 2003 (F1 %)
Collobert et al. [5]	MLP with word embeddings+gazetteer	89.59
Passos et al. [149]	Lexicon Infused Phrase Embeddings	90.90
Chiu and Nichols [150]	Bi-LSTM with word+char+lexicon embeddings	90.77
Luo et al. [151]	Semi-CRF jointly trained with linking	91.20
Lample et al. [93]	Bi-LSTM-CRF with word+char embeddings	90.94
Lample et al. [93]	Bi-LSTM with word+char embeddings	89.15
Strubell et al. [152]	Dilated CNN with CRF	90.54

C. Named-Entity Recognition

CoNLL 2003 has been a standard English dataset for NER, which concentrates on four types of named entities: people, locations, organizations and miscellaneous entities. NER is one of the NLP problems where lexicons can be very useful. Collobert et al. [5] first achieved competitive results with neural structures augmented by gazetteer features. Chiu and Nichols [150] concatenated lexicon features, character embeddings and word embeddings and fed them as input to a bidirectional LSTM. On the other hand, Lample et al. [93] only relied on character and word embeddings, with pre-training embeddings on large unsupervised corpora, they achieved competitive results without using any lexicon. Similar to POS tagging, CRF also boosted the performance of NER, as demonstrated by the comparison in [93]. Overall, we see that bidirectional LSTM with CRF acts as a strong model for NLP problems related to structured prediction.

Passos et al. [149] proposed to modify skip-gram models to better learn entity-type related word embeddings that can leverage information from relevant lexicons. Luo et al. [151] jointly optimized the entities and the linking of entities to a KB. Strubell et al. [152] proposed to use dilated convolutions, defined over a wider effective input width by skipping over certain inputs at a time, for better parallelization and context modeling. The model showed significant speedup while retaining accuracy.

D. Semantic Role Labeling

Semantic role labeling (SRL) aims to discover the predicate-argument structure of each predicate in a sentence. For each target verb (predicate), all constituents in the sentence which take a semantic role of the verb are recognized. Typical semantic arguments include Agent, Patient, Instrument, etc., and also adjuncts such as Locative, Temporal, Manner, Cause, etc. [153]. Table V shows the performance of different models on the CoNLL 2005 and 2012 datasets.

Traditional SRL systems consist of several stages: producing a parse tree, identifying which parse tree nodes represent the arguments of a given verb, and finally classifying these nodes to determine the corresponding SRL tags. Each classification process usually entails extracting numerous features and feeding them into statistical models [5].

Given a predicate, Täckström et al. [154] scored a constituent span and its possible role to that predicate with a series of features based on the parse tree. They proposed a dynamic programming algorithm for efficient inference. Collobert et al. [5] achieved comparable results with a convolution neural networks augmented by parsing information provided in the form of additional look-up tables. Zhou and Xu [153] proposed to use bidirectional LSTM to model arbitrarily long context, which proved to be successful without any parsing tree information. He et al. [155] further extended this work by introducing highway connections [156], more advanced regularization and ensemble of multiple experts.

TABLE V: **Semantic Role Labeling**

Paper	Model	CoNLL2005 (F1 %)	CoNLL2012 (F1 %)
Collobert et al. [5]	CNN with parsing features	76.06	
Täckström et al. [154]	Manual features with DP for inference	78.6	79.4
Zhou and Xu [153]	Bidirectional LSTM	81.07	81.27
He et al. [155]	Bidirectional LSTM with highway connections	83.2	83.4

E. Sentiment Classification

The Stanford Sentiment Treebank (SST) dataset contains sentences taken from the movie review website Rotten Tomatoes. It was proposed by Pang and Lee [157] and subsequently extended by Socher et al. [4]. The annotation scheme has inspired a new dataset for sentiment analysis, called CMU-MOSI, where sentiment is studied in a multimodal setup [158].

[4] and [119] were both recursive networks that relied on constituency parsing trees. Their difference shows the effectiveness of LSTM over vanilla RNN in modeling sentences. On the other hand, tree-LSTM performed better than linear bidirectional LSTM, implying that tree structures can potentially better capture the syntactical property of natural sentences. Yu et al. [159] proposed to refine pre-trained word embeddings with a sentiment lexicon, observing improved results based on [119].

Kim [50] and Kalchbrenner et al. [49] both used convolutional layers. The model [50] was similar to the one in Fig. 6, while Kalchbrenner et al. [49] constructed the model in a hierarchical manner by interweaving k-max pooling layers with convolutional layers.

TABLE VI: **Sentiment Classification** (SST-1 = Stanford Sentiment Treebank, fine-grained 5 classes Socher et al. [4]; SST-2: the binary version of SST-1; Numbers are accuracies (%))

Paper	Model	SST-1	SST-2
Socher et al. [4]	Recursive Neural Tensor Network	45.7	85.4
Kim [50]	Multichannel CNN	47.4	88.1
Kalchbrenner et al. [49]	DCNN with k-max pooling	48.5	86.8
Tai et al. [119]	Bidirectional LSTM	48.5	87.2
Le and Mikolov [160]	Paragraph Vector	48.7	87.8
Tai et al. [119]	Constituency Tree-LSTM	51.0	88.0
Yu et al. [159]	Tree-LSTM with refined word embeddings	54.0	90.3
Kumar et al. [102]	DMN	52.1	88.6

F. Machine Translation

The phrase-based SMT framework [161] factorized the translation model into the translation probabilities of matching phrases in the source and target sentences. Cho et al. [82] proposed to learn the translation probability of a source phrase to a corresponding target phrase with an RNN encoder-decoder. Such a scheme of scoring phrase pairs improved translation performance. Sutskever et al. [74], on the other hand, re-scored the top 1000 best candidate translations produced by an SMT system with a 4-layer LSTM seq2seq model. Dispensing the traditional SMT system entirely, Wu et al. [162] trained a deep LSTM network with 8 encoder and 8 decoder layers with residual connections as well as attention connections. Wu et al. [162] then refined the model by using reinforcement learning to directly optimize BLEU scores, but they found that the improvement in BLEU scores by this method did not reflect in human evaluation of translation quality. Recently, Gehring et al. [163] proposed a CNN-based seq2seq learning model for machine translation. The representation for each word in the input is computed by CNN in a parallelized style for the attention mechanism. The decoder state is also determined by CNN with words that are already produced. Vaswani et al. [113] proposed a self-attention-based model and dispensed convolutions and recurrences entirely.

TABLE VII: **Machine translation** (Numbers are BLEU scores)

Paper	Model	WMT2014 English2German	WMT2014 English2French
Cho et al. [82]	Phrase table with neural features		34.50
Sutskever et al. [74]	Reranking phrase-based SMT best list with LSTM seq2seq		36.5
Wu et al. [162]	Residual LSTM seq2seq + Reinforcement learning refining	26.30	41.16
Gehring et al. [163]	seq2seq with CNN	26.36	41.29
Vaswani et al. [113]	Attention mechanism	28.4	41.0

G. Question answering

QA problems take many forms. Some rely on large KBs to answer open-domain questions, while others answer a question based on a few sentences or a paragraph (reading comprehension). For the former, we list (see Table VIII) several experiments conducted on a large-scale QA dataset introduced by [164], where 14M commonsense knowledge triples are considered as the KB. Each question can be answered with a single-relation query. For the latter, we consider (see Table VIII) (1) the synthetic dataset of *bAbI* [165], which requires the model to reason over multiple related facts to produce the right answer. It contains 20 synthetic tasks that test a model's ability to retrieve relevant facts and reason over them. Each task focuses on a different skill such as *basic coreference* and *size reasoning*. (2) The Stanford Question Answering Dataset (*SQuAD*) [166], consisting of 100,000+ questions posed by crowdworkers on a set of Wikipedia articles. The answer to each question is a segment of text from the corresponding article.

The central problem of learning to answer single-relation queries is to find the single supporting fact in the database. Fader et al. [164] proposed to tackle this problem by learning a lexicon that maps natural language patterns to database concepts

(entities, relations, question patterns) based on a question paraphrasing dataset. Bordes et al. [167] embedded both questions and KB triples as dense vectors and scored them with inner product.

Weston et al. [112] took a similar approach by treating the KB as long-term memory, while casting the problem in the framework of a memory network. On the *bAbI* dataset, Sukhbaatar et al. [138] improved upon the original memory networks model [112] by making the training procedure agnostic of the actual supporting fact, while Kumar et al. [102] used neural sequence models (GRU) instead of neural bag-of-words models as in [138] and [112] to embed memories.

For models on the *SQuAD* dataset, the goal is to determine the start point and end point of the answer segment. Chen et al. [168] encoded both the question and the words in context using LSTMs and used a bilinear matrix for calculating the similarity between the two. Shen et al. [169] proposed Reasonet, a model that read a document repeatedly with attention on different parts each time until a satisfying answer is found. Yu et al. [170] replaced RNNs with convolution and self-attention for encoding the question and the context with significant speed improvement.

TABLE VIII: **Question answering**

Paper	Model	bAbI (Mean accuracy %)	Farbes (Accuracy %)	SQuAD (EM/F1 %)
Fader et al. [164]	Paraphrase-driven lexicon learning		0.54	
Bordes et al. [167]	Weekly supervised embedding		0.73	
Weston et al. [112]	Memory networks	93.3	0.83	
Sukhbaatar et al. [138]	End-to-end memory networks	88.4		
Kumar et al. [102]	DMN	93.6		
Chen et al. [168]	Document Reader			70.0/79.0
Shen et al. [169]	ReasoNet			69.1/78.9
Yu et al. [170]	QAnet			76.2/84.6

H. Dialogue Systems

Two types of dialogue systems have been developed: generation-based models and retrieval-based models.

In Table IX, the Twitter Conversation Triple Dataset is typically used for evaluating generation-based dialogue systems, containing 3-turn Twitter conversation instances. One commonly used evaluation metric is BLEU [171], although it is commonly acknowledged that most automatic evaluation metrics are not completely reliable for dialogue evaluation and additional human evaluation is often necessary. Ritter et al. [172] employed the phrase-based statistical machine translation (SMT) framework to "translate" the message to its appropriate response. Sordoni et al. [173] reranked the 1000 best responses produced by SMT with a context-sensitive RNN encoder-decoder framework, observing substantial gains. Li et al. [174] reported results on replacing the traditional maximum log likelihood training objective with the maximum mutual information training objective, in an effort to produce interesting and diverse responses, both of which are tested on a 4-layer LSTM encoder-decoder framework.

The response retrieval task is defined as selecting the best response from a repository of candidate responses. Such a model can be evaluated by the recall1@k metric, where the ground-truth response is mixed with $k-1$ random responses. The Ubuntu dialogue dataset was constructed by scraping multi-turn Ubuntu trouble-shooting dialogues from an online chatroom [97]. Lowe et al. [97] used LSTMs to encode the message and response, and then inner product of the two sentence embeddings is used to rank candidates.

Zhou et al. [175] proposed to better exploit the multi-turn nature of human conversation by employing the LSTM encoder on top of sentence-level CNN embeddings, similar to [176]. Dodge et al. [177] cast the problem in the framework of a memory network, where the past conversation was treated as memory and the latest utterance was considered as a "question" to be responded to. The authors showed that using simple neural bag-of-word embedding for sentences can yield competitive results.

TABLE IX: **Dialogue systems**

Paper	Model	Twitter Conversation Triple Dataset (BLEU)	Ubuntu Dialogue Dataset (recall 1@10 %)
Ritter et al. [172]	SMT	3.60	
Sordoni et al. [173]	SMT+neural reranking	4.44	
Li et al. [174]	LSTM seq2seq	4.51	
Li et al. [174]	LSTM seq2seq with MMI objective	5.22	
Lowe et al. [97]	Dual LSTM encoders for semantic matching		55.22
Dodge et al. [177]	Memory networks		63.72
Zhou et al. [175]	Sentence-level CNN-LSTM encoder		66.15

I. Contextual Embeddings

In this section, we explore some of the recent results based on contextual embeddings as explained in section II-D. ELMo has contributed significantly towards the recent advancement of NLP. In various NLP tasks, ELMo outperformed state of the art by significant margin (Table X). However, latest mode BERT surpass ELMo to establish itself as the state-of-the-art in multiple tasks as summarized in Table XI.

TABLE X: Comparison of ELMo + Baseline with the previous state of the art (SOTA) on various NLP tasks. The table has been adapted from [41]. SOTA results have been taken from [41]; SQUAD [166]: QA task; SNLI [178]: Stanford Natural Language Inference task; SRL [153]: Semantic Role Labelling; Coref [179]: Coreference Resolution; NER [180]: Named Entity Recognition; SST-5 [4]: Stanford Sentiment Treebank 5-class classification;

Task	**Previous SOTA**	**Previous SOTA Results**	**Baseline**	**ELMo + Baseline**	**Increase (Absolute/Relative)**
SQuAD	Liu et al. [181]	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Qian et al. [182]	88.6	88.0	88.70 $\pm$ 0.17	0.7 / 5.8%
SRL	Luheng et al. [183]	81.7	81.4	84.6	3.2 / 17.2%
Coref	Kenton et al. [184]	67.2	67.2	70.4	3.2 / 9.8%
NER	Matthew et al. [185]	91.93 $\pm$ 0.19	90.15	92.22 $\pm$ 0.10	2.06 / 21%
SST-5	Bryan et al. [186]	53.7	51.4	54.7 0.5	3.3 / 6.8%

Task	**BiLSTM+ ELMo+Attn**	**BERT**
QNLI	79.9	91.1
SST-2	90.9	94.9
STS-B	73.3	86.5
RTE	56.8	70.1
SQuAD	85.8	91.1
NER	92.2	92.8

TABLE XI: QNLI [187]: Question Natural Language Inference task; SST-2 [4]: Stanford Sentiment Treebank binary classification; STS-B [188]: Semantic Textual Similarity Benchmark; RTE [189]: Recognizing Textual Entailment; SQUAD [166]: QA task; NER [180]: Named Entity Recognition.

IX. Conclusion

Deep learning offers a way to harness large amount of computation and data with little engineering by hand [90]. With distributed representation, various deep models have become the new state-of-the-art methods for NLP problems. Supervised learning is the most popular practice in recent deep learning research for NLP. In many real-world scenarios, however, we have unlabeled data which require advanced unsupervised or semi-supervised approaches. In cases where there is lack of labeled data for some particular classes or the appearance of a new class while testing the model, strategies like zero-shot learning should be employed. These learning schemes are still in their developing phase but we expect deep learning based NLP research to be driven in the direction of making better use of unlabeled data. We expect such trend to continue with more and better model designs. We expect to see more NLP applications that employ reinforcement learning methods, e.g., dialogue systems. We also expect to see more research on multimodal learning [190] as, in the real world, language is often grounded on (or correlated with) other signals.

Finally, we expect to see more deep learning models whose internal memory (bottom-up knowledge learned from the data) is enriched with an external memory (top-down knowledge inherited from a KB). Coupling symbolic and sub-symbolic AI will be key for stepping forward in the path from NLP to natural language understanding. Relying on machine learning, in fact, is good to make a 'good guess' based on past experience, because sub-symbolic methods encode correlation and their decision-making process is probabilistic. Natural language understanding, however, requires much more than that. To use Noam Chomsky's words, "you do not get discoveries in the sciences by taking huge amounts of data, throwing them into a computer and doing statistical analysis of them: that's not the way you understand things, you have to have theoretical insights".

References

[1] E. Cambria and B. White, "Jumping NLP curves: A review of natural language processing research," *IEEE Computational Intelligence Magazine*, vol. 9, no. 2, pp. 48–57, 2014.

[2] T. Mikolov, M. Karafiát, L. Burget, J. Cernockỳ, and S. Khudanpur, "Recurrent neural network based language model." in *Interspeech*, vol. 2, 2010, p. 3.

[3] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in neural information processing systems*, 2013, pp. 3111–3119.

[4] R. Socher, A. Perelygin, J. Y. Wu, J. Chuang, C. D. Manning, A. Y. Ng, C. Potts *et al.*, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of the conference on empirical methods in natural language processing (EMNLP)*, vol. 1631, 2013, p. 1642.

[5] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuksa, "Natural language processing (almost) from scratch," *Journal of Machine Learning Research*, vol. 12, no. Aug, pp. 2493–2537, 2011.

[6] Y. Goldberg, "A primer on neural network models for natural language processing," *Journal of Artificial Intelligence Research*, vol. 57, pp. 345–420, 2016.

[7] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *Journal of machine learning research*, vol. 3, no. Feb, pp. 1137–1155, 2003.

[8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[9] J. Weston, S. Bengio, and N. Usunier, "Wsabie: Scaling up to large vocabulary image annotation," in *IJCAI*, vol. 11, 2011, pp. 2764–2770.

[10] R. Socher, C. C. Lin, C. Manning, and A. Y. Ng, "Parsing natural scenes and natural language with recursive neural networks," in *Proceedings of the 28th international conference on machine learning (ICML-11)*, 2011, pp. 129–136.

[11] P. D. Turney and P. Pantel, "From frequency to meaning: Vector space models of semantics," *Journal of artificial intelligence research*, vol. 37, pp. 141–188, 2010.

[12] E. Cambria, S. Poria, A. Gelbukh, and M. Thelwall, "Sentiment analysis is a big suitcase," *IEEE Intelligent Systems*, vol. 32, no. 6, pp. 74–80, 2017.

[13] X. Glorot, A. Bordes, and Y. Bengio, "Domain adaptation for large-scale sentiment classification: A deep learning approach," in *Proceedings of the 28th international conference on machine learning (ICML-11)*, 2011, pp. 513–520.

[14] K. M. Hermann and P. Blunsom, "The role of syntax in vector space models of compositional semantics," in *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2013.

[15] J. L. Elman, "Distributed representations, simple recurrent networks, and grammatical structure," *Machine learning*, vol. 7, no. 2-3, pp. 195–225, 1991.

[16] A. M. Glenberg and D. A. Robertson, "Symbol grounding and meaning: A comparison of high-dimensional and embodied theories of meaning," *Journal of memory and language*, vol. 43, no. 3, pp. 379–401, 2000.

[17] S. T. Dumais, "Latent semantic analysis," *Annual review of information science and technology*, vol. 38, no. 1, pp. 188–230, 2004.

[18] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *Journal of machine Learning research*, vol. 3, no. Jan, pp. 993–1022, 2003.

[19] R. Collobert and J. Weston, "A unified architecture for natural language processing: Deep neural networks with multitask learning," in *Proceedings of the 25th international conference on Machine learning*. ACM, 2008, pp. 160–167.

[20] A. Gittens, D. Achlioptas, and M. W. Mahoney, "Skip-gram-zipf+ uniform= vector additivity," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, vol. 1, 2017, pp. 69–76.

[21] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation." in *EMNLP*, vol. 14, 2014, pp. 1532–1543.

[22] X. Rong, "word2vec parameter learning explained," *arXiv preprint arXiv:1411.2738*, 2014.

[23] R. Johnson and T. Zhang, "Semi-supervised convolutional neural networks for text categorization via region embedding," in *Advances in neural information processing systems*, 2015, pp. 919–927.

[24] R. Socher, J. Pennington, E. H. Huang, A. Y. Ng, and C. D. Manning, "Semi-supervised recursive autoencoders for predicting sentiment distributions," in *Proceedings of the conference on empirical methods in natural language processing*. Association for Computational Linguistics, 2011, pp. 151–161.

[25] X. Wang, Y. Liu, C. Sun, B. Wang, and X. Wang, "Predicting polarities of tweets by composing word embeddings with long short-term memory." in *ACL (1)*, 2015, pp. 1343–1353.

[26] D. Tang, F. Wei, N. Yang, M. Zhou, T. Liu, and B. Qin, "Learning sentiment-specific word embedding for twitter sentiment classification." in *ACL (1)*, 2014, pp. 1555–1565.

[27] I. Labutov and H. Lipson, "Re-embedding words." in *ACL (2)*, 2013, pp. 489–493.

[28] S. Upadhyay, K.-W. Chang, M. Taddy, A. Kalai, and J. Zou, "Beyond bilingual: Multi-sense word embeddings using multilingual context," *arXiv preprint arXiv:1706.08160*, 2017.

[29] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, "Character-aware neural language models." in *AAAI*, 2016, pp. 2741–2749.

[30] C. N. Dos Santos and M. Gatti, "Deep convolutional neural networks for sentiment analysis of short texts." in *COLING*, 2014, pp. 69–78.

[31] C. N. d. Santos and V. Guimaraes, "Boosting named entity recognition with neural character embeddings," *arXiv preprint arXiv:1505.05008*, 2015.

[32] C. D. Santos and B. Zadrozny, "Learning character-level representations for part-of-speech tagging," in *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, 2014, pp. 1818–1826.

[33] Y. Ma, E. Cambria, and S. Gao, "Label embedding for zero-shot fine-grained named entity typing," in *COLING*, Osaka, 2016, pp. 171–180.

[34] X. Chen, L. Xu, Z. Liu, M. Sun, and H. Luan, "Joint learning of character and word embeddings," in *Twenty-Fourth International Joint Conference on Artificial Intelligence*, 2015.

[35] X. Zheng, H. Chen, and T. Xu, "Deep learning for chinese word segmentation and pos tagging." in *EMNLP*, 2013, pp. 647–657.

[36] H. Peng, E. Cambria, and X. Zou, "Radical-based hierarchical embeddings for chinese sentiment analysis at sentence level," in *FLAIRS*, 2017, pp. 347–352.

[37] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *arXiv preprint*

arXiv:1607.04606, 2016.

[38] A. Herbelot and M. Baroni, "High-risk learning: acquiring new word vectors from tiny data," *arXiv preprint arXiv:1707.06556*, 2017.

[39] Y. Pinter, R. Guthrie, and J. Eisenstein, "Mimicking word embeddings using subword rnns," *arXiv preprint arXiv:1707.06961*, 2017.

[40] L. Lucy and J. Gauthier, "Are distributional representations ready for the real world? evaluating word vectors for grounded perceptual meaning," *arXiv preprint arXiv:1705.11168*, 2017.

[41] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," *arXiv preprint arXiv:1802.05365*, 2018.

[42] A. Mousa and B. Schuller, "Contextual bidirectional long short-term memory recurrent neural network language models: A generative approach to sentiment analysis," in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*, vol. 1, 2017, pp. 1023–1032.

[43] A. M. Dai and Q. V. Le, "Semi-supervised sequence learning," in *Advances in neural information processing systems*, 2015, pp. 3079–3087.

[44] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," *URL https://s3-us-west-2. amazonaws. com/openai-assets/research-covers/language-unsupervised/language_understanding_paper. pdf*, 2018.

[45] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[46] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[47] A. Sharif Razavian, H. Azizpour, J. Sullivan, and S. Carlsson, "Cnn features off-the-shelf: an astounding baseline for recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2014, pp. 806–813.

[48] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proceedings of the 22nd ACM international conference on Multimedia*. ACM, 2014, pp. 675–678.

[49] N. Kalchbrenner, E. Grefenstette, and P. Blunsom, "A convolutional neural network for modelling sentences," *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics*, June 2014. [Online]. Available: http://goo.gl/EsQCuC

[50] Y. Kim, "Convolutional neural networks for sentence classification," *arXiv preprint arXiv:1408.5882*, 2014.

[51] Y. Zhang and B. Wallace, "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification," *arXiv preprint arXiv:1510.03820*, 2015.

[52] S. Poria, E. Cambria, and A. Gelbukh, "Aspect extraction for opinion mining with a deep convolutional neural network," *Knowledge-Based Systems*, vol. 108, pp. 42–49, 2016.

[53] A. Kirillov, D. Schlesinger, W. Forkel, A. Zelenin, S. Zheng, P. Torr, and C. Rother, "Efficient likelihood learning of a generic cnn-crf model for semantic segmentation," *arXiv preprint arXiv:1511.05067*, 2015.

[54] A. Waibel, T. Hanazawa, G. Hinton, K. Shikano, and K. J. Lang, "Phoneme recognition using time-delay neural networks," *IEEE transactions on acoustics, speech, and signal processing*, vol. 37, no. 3, pp. 328–339, 1989.

[55] A. Mukherjee and B. Liu, "Aspect extraction through semi-supervised modeling," in *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics: Long Papers-Volume 1*. Association for Computational Linguistics, 2012, pp. 339–348.

[56] S. Ruder, P. Ghaffari, and J. G. Breslin, "Insight-1 at semeval-2016 task 5: Deep learning for multilingual aspect-based sentiment analysis," *arXiv preprint arXiv:1609.02748*, 2016.

[57] P. Wang, J. Xu, B. Xu, C.-L. Liu, H. Zhang, F. Wang, and H. Hao, "Semantic clustering and convolutional neural network for short text categorization." in *ACL (2)*, 2015, pp. 352–357.

[58] S. Poria, E. Cambria, D. Hazarika, and P. Vij, "A deeper look into sarcastic tweets using deep convolutional neural networks," in *COLING*, 2016, pp. 1601–1612.

[59] M. Denil, A. Demiraj, N. Kalchbrenner, P. Blunsom, and N. de Freitas, "Modelling, visualising and summarising documents with a single convolutional neural network," *26th International Conference on Computational Linguistics*, pp. 1601–1612, 2014.

[60] B. Hu, Z. Lu, H. Li, and Q. Chen, "Convolutional neural network architectures for matching natural language sentences," in *Advances in neural information processing systems*, 2014, pp. 2042–2050.

[61] Y. Shen, X. He, J. Gao, L. Deng, and G. Mesnil, "A latent semantic model with convolutional-pooling structure for information retrieval," in *Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management*. ACM, 2014, pp. 101–110.

[62] W.-t. Yih, X. He, and C. Meek, "Semantic parsing for single-relation question answering." in *ACL (2)*. Citeseer, 2014, pp. 643–648.

[63] L. Dong, F. Wei, M. Zhou, and K. Xu, "Question answering over freebase with multi-column convolutional neural networks." in *ACL (1)*, 2015, pp. 260–269.

[64] A. Severyn and A. Moschitti, "Modeling relational information in question-answer pairs with convolutional neural networks," *arXiv preprint arXiv:1604.01178*, 2016.

[65] Y. Chen, L. Xu, K. Liu, D. Zeng, J. Zhao *et al.*, "Event extraction via dynamic multi-pooling convolutional neural networks." in *ACL (1)*, 2015, pp. 167–176.

[66] O. Abdel-Hamid, A.-r. Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Transactions on audio, speech, and language processing*, vol. 22, no. 10, pp. 1533–1545, 2014.

[67] D. Palaz, R. Collobert *et al.*, "Analysis of cnn-based speech recognition system using raw speech as input," Idiap, Tech. Rep., 2015.

[68] Z. Tu, B. Hu, Z. Lu, and H. Li, "Context-dependent translation selection using convolutional neural network," *arXiv preprint arXiv:1503.02357*, 2015.

[69] J. L. Elman, "Finding structure in time," *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.

[70] T. Mikolov, S. Kombrink, L. Burget, J. Černockỳ, and S. Khudanpur, "Extensions of recurrent neural network language model," in *Acoustics, Speech and Signal Processing (ICASSP), 2011 IEEE International Conference on*. IEEE, 2011, pp. 5528–5531.

[71] I. Sutskever, J. Martens, and G. E. Hinton, "Generating text with recurrent neural networks," in *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, 2011, pp. 1017–1024.

[72] S. Liu, N. Yang, M. Li, and M. Zhou, "A recursive recurrent neural network for statistical machine translation," *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics*, pp. 1491–1500, 2014.

[73] M. Auli, M. Galley, C. Quirk, and G. Zweig, "Joint language and translation modeling with recurrent neural networks." in *EMNLP*, 2013, pp. 1044–1054.

[74] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in neural information processing systems*, 2014, pp. 3104–3112.

[75] T. Robinson, M. Hochberg, and S. Renals, "The use of recurrent neural networks in continuous speech recognition," in *Automatic speech and speaker recognition*. Springer, 1996, pp. 233–258.

[76] A. Graves, A.-r. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Acoustics, speech and signal processing (icassp), 2013 ieee international conference on*. IEEE, 2013, pp. 6645–6649.

[77] A. Graves and N. Jaitly, "Towards end-to-end speech recognition with recurrent neural networks," in *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, 2014, pp. 1764–1772.

[78] H. Sak, A. Senior, and F. Beaufays, "Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition," *arXiv preprint arXiv:1402.1128*, 2014.

[79] A. Karpathy and L. Fei-Fei, "Deep visual-semantic alignments for generating image descriptions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3128–3137.

[80] D. Tang, B. Qin, and T. Liu, "Document modeling with gated recurrent neural network for sentiment classification." in *EMNLP*, 2015, pp. 1422–1432.

[81] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[82] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[83] G. Chen, D. Ye, E. Cambria, J. Chen, and Z. Xing, "Ensemble application of convolutional and recurrent neural networks for multi-label text categorization," in *IJCNN*, 2017, pp. 2377–2383.

[84] S. Poria, E. Cambria, D. Hazarika, N. Mazumder, A. Zadeh, and L.-P. Morency, "Context-dependent sentiment analysis in user-generated videos," in *ACL*, 2017, pp. 873–883.

[85] A. Zadeh, M. Chen, S. Poria, E. Cambria, and L.-P. Morency, "Tensor fusion network for multimodal sentiment analysis," in *Empirical Methods in NLP*, 2017.

[86] E. Tong, A. Zadeh, and L.-P. Morency, "Combating human trafficking with deep multimodal models," in *Association for Computational Linguistics*, 2017.

[87] I. Chaturvedi, E. Ragusa, P. Gastaldo, R. Zunino, and E. Cambria, "Bayesian network based extreme learning machine for subjectivity detection," *Journal of The Franklin Institute*, 2017.

[88] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," *arXiv preprint arXiv:1612.08083*, 2016.

[89] W. Yin, K. Kann, M. Yu, and H. Schütze, "Comparative study of cnn and rnn for natural language processing," *arXiv preprint arXiv:1702.01923*, 2017.

[90] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[91] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[92] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with lstm," *9th International Conference on Artificial Neural Networks*, pp. 850–855, 1999.

[93] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," *arXiv preprint arXiv:1603.01360*, 2016.

[94] A. Graves, "Generating sequences with recurrent neural networks," *arXiv preprint arXiv:1308.0850*, 2013.

[95] M. Sundermeyer, H. Ney, and R. Schlüter, "From feedforward to recurrent lstm neural networks for language modeling," *IEEE/ACM Transactions on Audio, Speech and Language Processing (TASLP)*, vol. 23, no. 3, pp. 517–529, 2015.

[96] M. Sundermeyer, T. Alkhouli, J. Wuebker, and H. Ney, "Translation modeling with bidirectional recurrent neural networks." in *EMNLP*, 2014, pp. 14–25.

[97] R. Lowe, N. Pow, I. Serban, and J. Pineau, "The ubuntu dialogue corpus: A large dataset for research in unstructured multi-turn dialogue systems," *arXiv preprint arXiv:1506.08909*, 2015.

[98] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3156–3164.

[99] O. Vinyals and Q. Le, "A neural conversational model," *arXiv preprint arXiv:1506.05869*, 2015.

[100] J. Li, M. Galley, C. Brockett, G. P. Spithourakis, J. Gao, and B. Dolan, "A persona-based neural conversation model," *arXiv preprint arXiv:1603.06155*, 2016.

[101] M. Malinowski, M. Rohrbach, and M. Fritz, "Ask your neurons: A neural-based approach to answering questions about images," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1–9.

[102] A. Kumar, O. Irsoy, J. Su, J. Bradbury, R. English, B. Pierce, P. Ondruska, I. Gulrajani, and R. Socher, "Ask me anything: Dynamic memory networks for natural language processing," *CoRR, abs/1506.07285*, 2015.

[103] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[104] A. M. Rush, S. Chopra, and J. Weston, "A neural attention model for abstractive sentence summarization," *arXiv preprint arXiv:1509.00685*, 2015.

[105] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhudinov, R. Zemel, and Y. Bengio, "Show, attend and tell: Neural image caption generation with visual attention," in *International Conference on Machine Learning*, 2015, pp. 2048–2057.

[106] O. Vinyals, Ł. Kaiser, T. Koo, S. Petrov, I. Sutskever, and G. Hinton, "Grammar as a foreign language," in *Advances in Neural Information Processing Systems*, 2015, pp. 2773–2781.

[107] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems*, 2015, pp. 2692–2700.

[108] R. Paulus, C. Xiong, and R. Socher, "A deep reinforced model for abstractive summarization," *arXiv preprint arXiv:1705.04304*, 2017.

[109] Y. Wang, M. Huang, X. Zhu, and L. Zhao, "Attention-based lstm for aspect-level sentiment classification." in *EMNLP*, 2016, pp. 606–615.

[110] Y. Ma, H. Peng, and E. Cambria, "Targeted aspect-based sentiment analysis via embedding commonsense knowledge into an attentive lstm," in *AAAI*, 2018.

[111] D. Tang, B. Qin, and T. Liu, "Aspect level sentiment classification with deep memory network," *arXiv preprint arXiv:1605.08900*, 2016.

[112] J. Weston, S. Chopra, and A. Bordes, "Memory networks," *arXiv preprint arXiv:1410.3916*, 2014.

[113] A. Vaswani, N. Shazeer, N. Parmar, and J. Uszkoreit, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017.

[114] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[115] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[116] R. Socher, B. Huval, C. D. Manning, and A. Y. Ng, "Semantic compositionality through recursive matrix-vector spaces," in *Proceedings of the 2012 joint conference on empirical methods in natural language processing and computational natural language learning*. Association for Computational Linguistics, 2012, pp. 1201–1211.

[117] B. Taskar, C. Guestrin, and D. Koller, "Max-margin markov networks," in *Advances in neural information processing systems*, 2004, pp. 25–32.

[118] S. R. Bowman, C. Potts, and C. D. Manning, "Recursive neural networks can learn logical semantics," *arXiv preprint arXiv:1406.1827*, 2014.

[119] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," *arXiv preprint arXiv:1503.00075*, 2015.

[120] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Advances in Neural Information Processing Systems*, 2015, pp. 1171–1179.

[121] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, "Sequence level training with recurrent neural networks," *arXiv preprint arXiv:1511.06732*, 2015.

[122] J. Li, W. Monroe, A. Ritter, M. Galley, J. Gao, and D. Jurafsky, "Deep reinforcement learning for dialogue generation," *arXiv preprint arXiv:1606.01541*, 2016.

[123] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine*

learning, vol. 8, no. 3-4, pp. 229–256, 1992.
[124] S. Young, M. Gašić, S. Keizer, F. Mairesse, J. Schatzmann, B. Thomson, and K. Yu, "The hidden information state model: A practical framework for pomdp-based spoken dialogue management," *Computer Speech & Language*, vol. 24, no. 2, pp. 150–174, 2010.
[125] S. Young, M. Gašić, B. Thomson, and J. D. Williams, "Pomdp-based statistical spoken dialog systems: A review," *Proceedings of the IEEE*, vol. 101, no. 5, pp. 1160–1179, 2013.
[126] P.-h. Su, V. David, D. Kim, T.-h. Wen, and S. Young, "Learning from real users: Rating dialogue success with neural networks for reinforcement learning in spoken dialogue systems," in *in Proceedings of Interspeech*. Citeseer, 2015, pp. 2007–2011.
[127] P.-H. Su, M. Gasic, N. Mrksic, L. Rojas-Barahona, S. Ultes, D. Vandyke, T.-H. Wen, and S. Young, "On-line active reward learning for policy optimisation in spoken dialogue systems," *arXiv preprint arXiv:1605.07669*, 2016.
[128] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in neural information processing systems*, 2014, pp. 2672–2680.
[129] J. Li, W. Monroe, T. Shi, A. Ritter, and D. Jurafsky, "Adversarial learning for neural dialogue generation," *arXiv preprint arXiv:1701.06547*, 2017.
[130] R. Kiros, Y. Zhu, R. R. Salakhutdinov, R. Zemel, R. Urtasun, A. Torralba, and S. Fidler, "Skip-thought vectors," in *Advances in neural information processing systems*, 2015, pp. 3294–3302.
[131] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," DTIC Document, Tech. Rep., 1985.
[132] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
[133] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," *arXiv preprint arXiv:1511.06349*, 2015.
[134] Y. Zhang, Z. Gan, and L. Carin, "Generating text via adversarial training," in *NIPS workshop on Adversarial Training*, 2016.
[135] Z. Hu, Z. Yang, X. Liang, R. Salakhutdinov, and E. P. Xing, "Controllable text generation," *arXiv preprint arXiv:1703.00955*, 2017.
[136] L. Yu, W. Zhang, J. Wang, and Y. Yu, "Seqgan: sequence generative adversarial nets with policy gradient," in *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.
[137] S. Rajeswar, S. Subramanian, F. Dutil, C. Pal, and A. Courville, "Adversarial generation of natural language," *arXiv preprint arXiv:1705.10929*, 2017.
[138] S. Sukhbaatar, J. Weston, R. Fergus *et al.*, "End-to-end memory networks," in *Advances in neural information processing systems*, 2015, pp. 2440–2448.
[139] C. Xiong, S. Merity, and R. Socher, "Dynamic memory networks for visual and textual question answering," *arXiv*, vol. 1603, 2016.
[140] J. Giménez and L. Marquez, "Fast and accurate part-of-speech tagging: The svm approach revisited," *Recent Advances in Natural Language Processing III*, pp. 153–162, 2004.
[141] Z. Huang, W. Xu, and K. Yu, "Bidirectional lstm-crf models for sequence tagging," *arXiv preprint arXiv:1508.01991*, 2015.
[142] D. Andor, C. Alberti, D. Weiss, A. Severyn, A. Presta, K. Ganchev, S. Petrov, and M. Collins, "Globally normalized transition-based neural networks," *arXiv preprint arXiv:1603.06042*, 2016.
[143] D. Chen and C. D. Manning, "A fast and accurate dependency parser using neural networks." in *EMNLP*, 2014, pp. 740–750.
[144] C. Dyer, M. Ballesteros, W. Ling, A. Matthews, and N. A. Smith, "Transition-based dependency parsing with stack long short-term memory," *arXiv preprint arXiv:1505.08075*, 2015.
[145] H. Zhou, Y. Zhang, C. Cheng, S. Huang, X. Dai, and J. Chen, "A neural probabilistic structured-prediction method for transition-based natural language processing," *Journal of Artificial Intelligence Research*, vol. 58, pp. 703–729, 2017.
[146] M. Zhu, Y. Zhang, W. Chen, M. Zhang, and J. Zhu, "Fast and accurate shift-reduce constituent parsing." in *ACL (1)*, 2013, pp. 434–443.
[147] D. Weiss, C. Alberti, M. Collins, and S. Petrov, "Structured training for neural network transition-based parsing," *arXiv preprint arXiv:1506.06158*, 2015.
[148] S. Petrov, L. Barrett, R. Thibaux, and D. Klein, "Learning accurate, compact, and interpretable tree annotation," in *Proceedings of the 21st International Conference on Computational Linguistics and the 44th annual meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2006, pp. 433–440.
[149] A. Passos, V. Kumar, and A. McCallum, "Lexicon infused phrase embeddings for named entity resolution," *arXiv preprint arXiv:1404.5367*, 2014.
[150] J. P. Chiu and E. Nichols, "Named entity recognition with bidirectional lstm-cnns," *arXiv preprint arXiv:1511.08308*, 2015.
[151] G. Luo, X. Huang, C.-Y. Lin, and Z. Nie, "Joint named entity recognition and disambiguation," in *Proc. EMNLP*, 2015,

pp. 879–880.
[152] E. Strubell, P. Verga, D. Belanger, and A. McCallum, "Fast and accurate sequence labeling with iterated dilated convolutions," *arXiv preprint arXiv:1702.02098*, 2017.
[153] J. Zhou and W. Xu, "End-to-end learning of semantic role labeling using recurrent neural networks." in *ACL (1)*, 2015, pp. 1127–1137.
[154] O. Täckström, K. Ganchev, and D. Das, "Efficient inference and structured learning for semantic role labeling," *Transactions of the Association for Computational Linguistics*, vol. 3, pp. 29–41, 2015.
[155] L. He, K. Lee, M. Lewis, and L. Zettlemoyer, "Deep semantic role labeling: What works and what's next," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2017.
[156] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training very deep networks," in *Advances in neural information processing systems*, 2015, pp. 2377–2385.
[157] B. Pang and L. Lee, "Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales," in *Proceedings of the 43rd annual meeting on association for computational linguistics*. Association for Computational Linguistics, 2005, pp. 115–124.
[158] A. Zadeh, R. Zellers, E. Pincus, and L.-P. Morency, "Multimodal sentiment intensity analysis in videos: Facial gestures and verbal messages," *IEEE Intelligent Systems*, vol. 31, no. 6, pp. 82–88, 2016.
[159] L.-C. Yu, J. Wang, K. R. Lai, and X. Zhang, "Refining word embeddings for sentiment analysis," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 545–550.
[160] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, 2014, pp. 1188–1196.
[161] P. Koehn, F. J. Och, and D. Marcu, "Statistical phrase-based translation," in *Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology-Volume 1*. Association for Computational Linguistics, 2003, pp. 48–54.
[162] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey *et al.*, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.
[163] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," *arXiv preprint arXiv:1705.03122*, 2017.
[164] A. Fader, L. S. Zettlemoyer, and O. Etzioni, "Paraphrase-driven learning for open question answering." in *ACL (1)*, 2013, pp. 1608–1618.
[165] J. Weston, A. Bordes, S. Chopra, A. M. Rush, B. van Merriënboer, A. Joulin, and T. Mikolov, "Towards ai-complete question answering: A set of prerequisite toy tasks," *arXiv preprint arXiv:1502.05698*, 2015.
[166] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.
[167] A. Bordes, J. Weston, and N. Usunier, "Open question answering with weakly supervised embedding models," in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2014, pp. 165–180.
[168] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading wikipedia to answer open-domain questions," *arXiv preprint arXiv:1704.00051*, 2017.
[169] Y. Shen, P.-S. Huang, J. Gao, and W. Chen, "Reasonet: Learning to stop reading in machine comprehension," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2017, pp. 1047–1055.
[170] A. W. Yu, D. Dohan, M.-T. Luong, R. Zhao, K. Chen, M. Norouzi, and Q. V. Le, "Qanet: Combining local convolution with global self-attention for reading comprehension," *arXiv preprint arXiv:1804.09541*, 2018.
[171] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting on association for computational linguistics*. Association for Computational Linguistics, 2002, pp. 311–318.
[172] A. Ritter, C. Cherry, and W. B. Dolan, "Data-driven response generation in social media," in *Proceedings of the conference on empirical methods in natural language processing*. Association for Computational Linguistics, 2011, pp. 583–593.
[173] A. Sordoni, M. Galley, M. Auli, C. Brockett, Y. Ji, M. Mitchell, J.-Y. Nie, J. Gao, and B. Dolan, "A neural network approach to context-sensitive generation of conversational responses," *arXiv preprint arXiv:1506.06714*, 2015.
[174] J. Li, M. Galley, C. Brockett, J. Gao, and B. Dolan, "A diversity-promoting objective function for neural conversation models," *arXiv preprint arXiv:1510.03055*, 2015.
[175] X. Zhou, D. Dong, H. Wu, S. Zhao, D. Yu, H. Tian, X. Liu, and R. Yan, "Multi-view response selection for human-computer conversation." in *EMNLP*, 2016, pp. 372–381.
[176] I. V. Serban, A. Sordoni, Y. Bengio, A. C. Courville, and J. Pineau, "Building end-to-end dialogue systems using generative hierarchical neural network models." in *AAAI*, 2016, pp. 3776–3784.
[177] J. Dodge, A. Gane, X. Zhang, A. Bordes, S. Chopra, A. Miller, A. Szlam, and J. Weston, "Evaluating prerequisite qualities for learning end-to-end dialog systems," *arXiv preprint arXiv:1511.06931*, 2015.

[178] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," *arXiv preprint arXiv:1508.05326*, 2015.
[179] S. Pradhan, A. Moschitti, N. Xue, O. Uryupina, and Y. Zhang, "Conll-2012 shared task: Modeling multilingual unrestricted coreference in ontonotes," in *Joint Conference on EMNLP and CoNLL-Shared Task*. Association for Computational Linguistics, 2012, pp. 1–40.
[180] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the conll-2003 shared task: Language-independent named entity recognition," in *Proceedings of the seventh conference on Natural language learning at HLT-NAACL 2003-Volume 4*. Association for Computational Linguistics, 2003, pp. 142–147.
[181] X. Liu, Y. Shen, K. Duh, and G. Jian-feng, "Stochastic answer networks for machine reading comprehension," *arXiv preprint arXiv:1712.03556*, 2017.
[182] C. Qian, Z. Xiao-Dan, L. Zhen-Hua, W. Si, J. Hui, and I. Diana, "Enhanced lstm for natural language inference," *In ACL*, 2017.
[183] H. Luheng, L. Kenton, L. Mike, and S. Z. Luke, "Deep semantic role labeling: What works and whats next," *In ACL*, 2017.
[184] L. Kenton, H. Luheng, L. Mike, and S. Z. Luke, "End-to-end neural coreference resolution," *In EMNLP*, 2017.
[185] E. P. Matthew, A. Waleed, B. Chandra, and P. Russell, "Semi-supervised sequence tagging with bidirectional language models," *In ACL*, 2017.
[186] M. Bryan, B. James, X. Caiming, and S. Richard, "Learned in translation: Contextualized word vectors," *In NIPS 2017*, 2017.
[187] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.
[188] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation," *arXiv preprint arXiv:1708.00055*, 2017.
[189] L. Bentivogli, P. Clark, I. Dagan, and D. Giampiccolo, "The fifth pascal recognizing textual entailment challenge." in *TAC*, 2009.
[190] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *arXiv preprint arXiv:1705.09406*, 2017.